

Lab Manual — Notes for Teaching Assistants

DSA27 · Weeks 1 to 15 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026

Contents

1. The labs in one page	1
2. Running a session (2 hours)	3
3. What to demonstrate	4
4. Exercise notes	13
5. Checking students' work	40
6. Environment problems you will see	45
7. Answers to the take-home practice	46
8. Worked solution notes, labs 09–15	49

*This guide is for the TAs running the labs of weeks 1–15. It contains what to demonstrate, where students go wrong, how to check their work, the answers to the take-home practice and, in section 8, notes on the reference solutions. The solutions themselves are in **solutions/**, for students to use **after** they have tried. Weeks 1–3 teach Python in `labs/`; from Week 4 the lab is where students build that week's `dsa/` module.*

1. The labs in one page

Week	Lab	Core idea to land	The trap to demonstrate live
1	Interpreter, numbers, strings, lists	A name refers to an object	<code>b = a;</code> <code>b.append(4)</code> changes <code>a</code>
2	Control flow, functions, errors, modules	Functions return; errors are information	the mutable default <code>def f(a, L=[])</code>

Week	Lab	Core idea to land	The trap to demonstrate live
3	Data structures, classes, generators	Each structure has a cost; a class bundles data with its contract	the shared class variable <code>tricks = []</code>
4	Building a dynamic array	Size is not capacity; growing by a factor makes append amortised $O(1)$	a constant growth step: count the copies
5	Building a linked list	Change references in an order that never loses the rest of the list	<code>push_front</code> with its two assignments swapped: a node that points at itself
6	Stacks, brackets and postfix	The top goes where the $O(1)$ end is; the stack is the algorithm	the top-at-index-0 stack that passes every test
7	Queues and the ring buffer	Move the indices, not the data; % makes the block a ring	<code>head += 1</code> with no %, and full against empty
8	Eight ways to search	An invariant for <code>lo</code> and <code>hi</code> , tested on sizes 0, 1 and 2	<code>while lo < hi:</code> which tests fail, and why exactly those
9	Sorts you can watch	Write each sort once, as a generator; the plain sort, the animation and a comparison count all run the same code	<code>a[j] >= current</code> in insertion sort: it still sorts, and only the stability test notices
10	Divide, conquer, and a pivot	One small helper per sort (<code>_merge</code> , <code>_partition</code> , <code>_sift_down</code>); the pivot decides quicksort's shape	<code>pivot="first"</code> on sorted input, counted: $n(n-1)/2$ comparisons
11	Trees and the four walks	A BST is one invariant about whole subtrees; every operation walks one path, so the height is the cost	the two-children <code>delete</code> that always writes <code>successor.parent.left</code> : it passes the two-children test and breaks the root

Week	Lab	Core idea to land	The trap to demonstrate live
12	Heaps and priority queues	The array is the tree; every comparison goes through <code>_beats</code> ; <code>heapify</code> runs backwards in $O(n)$	sift-down that swaps with the first child that beats the item, not the better one
13	Hash tables, twice	The index is computed, not searched; a probe stops only at a never-used slot	an open-addressing delete that writes <code>None</code> : one test fails, and a key still in the table cannot be found
14	Graphs and the two searches	One loop, one choice: a queue gives breadth first, a stack gives depth first; the visited set is what makes both end	<code>dfs_iterative</code> that marks on push: every search test passes, and only <code>test_dfs_and_dfs_iterative</code> fails
15	A calculator, stage by stage	Precedence is the shape of the grammar; one method per rule, and the old tree goes on the left	the right-recursive <code>expr</code> : <code>1 - 2 - 3</code> gives 2, and only 3 of 55 tests notice

Each lab has a reading (`docs/labs/labNN-*.md`, also as a PDF in `docs/pdf/`), a starter file and a test file: `labs/labNN.py` and `tests/test_labNN.py` for weeks 1–3, then that week's `dsa/` module and its test file — the lab manual's summary table names both, and the `pytest -k` filter when the file is shared.

Where students are coming from. AI students have taken CS1002 Object Oriented Programming. Bio and SWE students have taken CS012, structured programming only, often in C++. Expect the Bio and SWE groups to struggle more in Lab 03 (classes) and the AI group to be over-confident in Lab 01 (they will get the floor-division and aliasing checkpoints wrong anyway). Pair a stronger student with a weaker one for the Checkpoints if the room allows it.

2. Running a session (2 hours)

Time	What	Notes
0:00–0:10	Attendance; environment problems	Attendance counts toward the 75% rule — record it every session.
0:10–0:35	Live demonstration at the REPL	Follow section 3 below. Project a terminal, large font. Type; do not paste.
0:35–1:40	Students work: Checkpoints, then exercises	Walk the room. Ask “what did you predict?” before “what did you get?”
1:40–2:00	Check-off	Each student shows <code>pytest tests/test_labNN.py</code> and explains one line you choose.

Principles for helping.

- Ask them to **read the failing test’s output aloud** before you say anything. Half the time they then see it.
- Point to the **docstring** and the **test file** before you give a hint. The habit of reading the specification is the lesson.
- Give the smallest hint that unblocks — the hints in each lab’s “Exercises” part are graded from small to large. Do not type on a student’s keyboard.
- When a student says “it works on my machine”, run the tests. The tests define “works”.

3. What to demonstrate

Lab 01 — Week 1

1. Start the REPL; `2 + 2; _; type(5), type(5.0); 8 / 4` is `2.0`.
2. `-7 // 2` and `-7 % 2`. Ask the room first; most will say `-3` and `-1`.
3. `2 ** 100`, then `0.1 + 0.2 == 0.3`. One sentence on why: binary fractions.
4. `word = 'Python'`; index, negative index, slice; draw the “edges between characters” diagram on the board; `word[:-1]`.
5. **The main event.** `a = [1, 2, 3]; b = a; b.append(4); a`. Then `a is b, id(a), c = a[:], c is a, c == a`. Draw the arrows on the board.
6. `[[0] * 3] * 3` and set one cell. Leave it unresolved — Lab 03 fixes it.
7. Open `labs/lab01.py`, write `celsius_to_fahrenheit` **wrongly** (no `round`), run `pytest tests/test_lab01.py -k celsius`, read the failure together, fix it. This is the only exercise you solve in front of them.

Lab 02 — Week 2

1. `if/elif` order: the score `>= 50` before `score >= 90` example.

2. `0` or `'default'`; `[]` and `1/0` — short-circuiting.
3. `for` over a string; `range(5)` is not a list; `enumerate`.
4. The prime-search with `for ... else`. Ask: which statement does the `else` belong to?
5. **The main event.** `def f(a, L=[])` called three times. Then the `None` fix. Then `add_item` versus `replace` — mutation versus rebinding.
6. Run `trace_demo.py` (Part 6.2) and read the traceback bottom-up, aloud.
7. `count_single / count_nested` for `n = 10, 100, 1000` — tie it to Lecture 02.

Lab 03 — Week 3

1. `marks.sort()` returns `None`; `sorted` does not. Everyone has hit this.
2. A list as a stack; then `pop(0)` versus `deque.popleft()`. Optional: time 100,000 `pop(0)` against 100,000 `popleft()`.
3. A comprehension next to the loop it replaces. Then the correct grid: `[[0] * 3 for _ in range(3)]` — closing the loop from Lab 01.
4. `{}` is a dict; `set()` is a set. `x in list` versus `x in set` on a million items with `time.perf_counter`.
5. The counting pattern with `get`, then `setdefault`.
6. **The main event.** Build `Dog` live; then `BadDog` with `tricks = []`. Then `Fraction` with only `__init__` and show `Fraction(1, 2)` prints as `<__main__.Fraction object at 0x...>`; add `__repr__` and show the difference; add `__eq__` and show `==` start working.
7. A three-line generator and `next()` by hand until `StopIteration`.

Lab 04 — Week 4

1. `from dsa.array import Array; a = Array(4); a[4] = 'e'` — the fixed size is the whole problem. Then `a[-1]`: the course `Array` has no negative indices, so the dynamic array must provide them.
2. Draw capacity against size on the board: a block of 8 with 5 used. Run the `show(arr)` helper from Part 0.5 of the manual after each of five appends, and ask the room for the capacity *before* each one appears.
3. Write `_resize` live (it is printed in the lecture): `allocate`, `copy`, `switch`, `count`. Then write `append` **wrongly** — write before the “is it full?” check — run `pytest tests/test_dynamic_array.py -k "constructor or grows_beyond or doubling"`, read `Array` index 1 out of range for length 1 aloud, and fix it. This is the only method you write in front of them.
4. `DynamicArray(growth=3)` and `growth=10`: print `resize_count` after 1,000 appends. Then `DynamicArray([1, 2], growth=1)` — ask why it fails.
5. The `StepArray` subclass (+ 10). Print `resize_count` for 1,000 appends (100 against 10), then run the Part 6 timing for `n = 500 ... 4,000` and show the two curves bend apart. Do not go bigger live: every element moves through the course `Array`’s bounds check, and the constant-step run grows fourfold per doubling.
6. `pop(0)` against `pop()` on 2,000 elements, timed (about 2 s against a few milliseconds; 8,000 takes tens of seconds per run). One sentence: “this is the queue you build in

Week 7".

Lab 05 — Week 5

1. Build `a = Node(3, Node(7, Node(1)))` at the REPL, from the end backwards. Walk it with `a.next.next.value`; draw the boxes and arrows on the board and label each arrow with the expression that reaches it. Then `draw_linked_list([3, 7, 1], highlight=1)` in the notebook.
2. **The main event.** `push_front` both ways. Write the correct two lines, then swap them in a subclass, push 3 and 7, and show `ll.head.next` is `ll.head` and `list(itertools.islice(ll, 5))` giving `[7, 7, 7, 7, 7]`. Then `list(ll)` and `Ctrl+C`. The rule: link the new node in before you let go of the old one.
3. `remove` on `3 → 7 → 1 → 9` at the whiteboard: the loop looks at `prev.next`, so it stays on the node before. Then ask the room how to remove the 3 — there is no `prev`: the head special case.
4. Trace `reverse` on `1 → 2 → 3 → 4` with three coloured markers (`prev`, `node`, the saved `next`). After each step, draw the **two** chains separately. Ask what happens if the saved-`next` line is dropped.
5. Time for `i in range(len(ll)): ll[i]` against for `v in ll` for `n = 1000` and `2000` with `time.perf_counter` (lists built with `push_front`). Four times against twice. "An interface does not tell you the cost."
6. Close with a stack: `push_front` three brackets, `pop_front` them back in reverse — the bridge to Lecture 06.

Lab 06 — Week 6

1. `pytest tests/test_stack_queue.py -k stack --collect-only -q` — it collects **all 18** tests, queues included, because `-k` also matches the file name `test_stack_queue`. Then show the filter the lab and Lecture 06 use, `-k "lifo or peek or emptybehaviour or balanced or infix"` (12 tests). Two minutes that save the room an hour of chasing queue failures.
2. Write `push` and `pop` live on the `DynamicArray`, `top` at the **end**. Then write `FrontStack(insert_at(0, ...) and pop(0))`, run the three stack tests on it — they pass — and ask the room what is wrong with it. Run Part 2's measurement with sizes up to 1,600 only: the index-0 line quadruples per doubling.
3. Whiteboard: the three unbalanced inputs `([])`, `)` and `(`, and which check each one needs. Leave the code to them.
4. Whiteboard: shunting-yard on `8 - 3 - 2` with two columns, output and operator stack. Stop at the second `-` and ask: does the first `-` leave? Then evaluate the postfix both ways (3 against 7) to show why it must — and point out the lecture's rule 2 ends "stopping at a `(`".
5. `8 3 -` at the board: `pop`, `pop`, and which one is the left operand. Name them right and left on the board, in the order they are popped.
6. Show `'3+4'.split()` at the REPL: one token. One sentence on why Week 15 starts with a tokenizer.

Lab 07 — Week 7

1. `SlowQueue` in two lines at the REPL, then time one `pop(0)` on a `DynamicArray` of 1,000 and of 4,000 items. Ask the room why it is four times slower, and why swapping the ends does not help.
2. **The main event.** On the whiteboard, draw a 4-slot block and run Lecture 07's capacity-4 trace (A to F) with the room calling out `head`, `size` and the tail (`head + size`) % 4 after each step. Stop at "enqueue E" and ask where it goes before you write it.
3. Show the start state and the full state side by side: `head == tail` in both. Ask how the class tells them apart; point at `is_empty` and `is_full`.
4. Read the given `CircularQueue.__iter__` aloud — `(head + offset) % capacity` — and say that `enqueue`, `dequeue` and the growth copy all reuse that formula.
5. Show the -k pitfall: `pytest tests/test_stack_queue.py -k queue --collect-only -q` collects all 18 tests, because -k also matches the file name `test_stack_queue.py` — so no filter may contain "queue" or "stack". Then the filter the lab and Lecture 07 use: `-k "fifo or dequeue or circular"` (6 tests).
6. The growth trap on the board: the full ring E F C D, head 2, copied slot by slot into 8 slots — read it from head 2 and let the room find E and F "lost". Then the unrolled copy. Do not write the code.

Lab 08 — Week 8

Midterm week: keep the demonstration to 20 minutes and the room on Parts 1–5.

1. On the board, trace `binary_search([4, 9, 13, 20, 26, 31, 37, 44], 10)` as a lo/hi/mid table. Stop on the last row (`lo = 2`, `hi = 1`) and ask what the 2 means: the insertion point, which is `lower_bound`.
2. Ask the room for `binary_search([3, 8], 8)` by hand, loop turns included. Then say the rule: test sizes 0, 1 and 2 before running `pytest`.
3. **The main event.** With a working `binary_search` projected, change `lo <= hi` to `lo < hi` and ask which of the 11 tests from `pytest tests/test_searching.py -v -k "binary_search and not recursive"` will fail. Collect guesses, then run it: exactly 4 fail and 7 pass. The 4 are target 9 in `[1, 3, 5, 7, 9]`, the empty/single test, first/last, and the million-element test. The three outside-the-values tests pass because they are misses, and the random-data test passes because only 2 of its 30 targets are present. Put the `=` back.
4. Change `lo = mid + 1` to `lo = mid` and run again: the run hangs on one test. Show `Ctrl+C` and read the traceback — it points at the `while` line.
5. Write `[1, 2, 2, 2, 5]` on the board with L/R marks for `< 2` and for `<= 2`: `lower_bound` and `upper_bound` are the same search for the first R. Say "half-open" and why `lo < hi` is right **there**.
6. Warn about the filter trap: `-k linear` selects 8 tests; `-k "linear_search and not agrees"` selects the 2 for `linear_search`.

Lab 09 — Week 9

Use the lab's list [4, 1, 3, 9, 7, 2] on the board all session — the students' Checkpoints 2 to 4 use it, so your drawings are their answer key.

1. Open notebooks/09-sorting-basic.ipynb on the projector and run the "Try it" cell. Step the slider slowly and ask, before each click, which pair is compared next and whether it swaps. Then, in a new cell, `frames = list(bubble_sort_steps([5, 2, 9, 1, 7]))` and `frames[0]`: the first frame is already sorted. Ask why (the notebook's version yields the same list every time; `list(...)` keeps eleven references to it). That is the reason for `list(a)` in every yield.
2. On the board, bubble sort on [4, 1, 3, 9, 7, 2], one row per pass, the room calling out the swaps: 4 + 1 + 1 + 1 + 0 swaps, 15 comparisons, all five passes. Count the inversions with them (7) and point out that it equals the swaps.
3. Selection sort on the same list, three swaps. Then the cards 2a, 2b, 1c: round 0 swaps 1c with 2a, and 2a lands behind 2b. Say the word **stable** and write the rule on the board: "moves past only strictly greater values".
4. Insertion sort on the same list: 7 shifts, 11 comparisons. Ask what the 7 is. Then write `j >= 0` and `a[j] > current` and ask what happens with the two halves swapped: on a list `a[-1]` silently reads the last element, on the course `Array` it raises.
5. **The main event: the price of watching.** Time the plain `bubble_sort` built on a generator that copies each frame, at `n = 100, 200` and `400`, with the lab's Part 7.1 snippet. Let the room predict the ratio first (most say 4). They see about 8. Ask where the extra `n` comes from, and show the snapshot fix: the ratio drops to about 4.
6. **Write one method live, wrongly, and fix it.** Write `insertion_sort_steps` with `while j >= 0` and `a[j] >= current`: and run `pytest tests/test_sorting.py -v -k insertion`. Twelve pass; only `test_insertion_sort_is_stable` fails, with `assert ['0d', '0b', '1c', '1a'] == ['0b', '0d', '1a', '1c']`. Read the test's `Card` class aloud: cards compare **by key only**, which is why the test can see stability at all — plain tuples would compare by their label too and hide the bug. Change `>=` to `>`: 13 passed.
7. Warn about the filter: `pytest tests/test_sorting.py` alone runs Week 10's merge, quick and heap tests too, which fail with `NotImplementedError`. Every command in the lab has a `-k` filter.

Lab 10 — Week 10

1. On the board, merge [2, 5, 9] with [1, 5, 6] (Checkpoint 1) with the room calling out `i`, `j`, `k` and the value copied. Stop at the tie, 5 against 5, and ask which one goes first and why it matters. Say the rule: `<=` takes from the left, and that one character is merge sort's stability.
2. Draw the merge tree of [6, 5, 3, 1, 8, 7, 2, 4] and number the merges in the order the recursion does them: left half completely first. Ask how many merges (`7 = n - 1`).
3. Trace Lomuto's partition of [6, 3, 9, 1, 8, 2, 7, 4] (Lecture 10's figure) with three colours of chalk: green `< pivot`, amber `≥ pivot`, white not yet seen. Say the

invariant aloud after every step.

4. **The main event.** With a working quicksort projected, and a Counted class like Lab 10, Part 6, ask the room to predict `comparisons(quick_sort, list(range(n)), "first")` for $n = 100, 200, 400$. Collect guesses; run it: 4,950, 19,900, 79,800 — exactly $n(n-1)/2$, times four per doubling. Then "median3": 606, 1,407, 3,208. Draw the two recursion shapes (Lecture 10, "The worst case, measured").
5. Write the inner sort **live, wrongly**: partition, then two recursive calls (`if lo < hi: ... yield from sort(lo, q - 1) ... yield from sort(q + 1, hi)`). Run `pytest tests/test_sorting.py -q -k quick: 13 pass`. Then run `quick_sort(list(range(1000)), pivot="first")` at the REPL: Recursion-Error. Ask why the tests did not see it (200 elements, default "median3"). Fix it live: while `lo < hi`:, recurse into the smaller part, move `lo` or `hi` for the larger. Run both again.
6. Draw `[2, 9, 4, 7, 1, 8, 5, 3]` as a tree with `draw_array_as_tree` projected; build the heap from index 3 down to 0 on the board. Ask why the loop cannot run upwards; if nobody answers, run it upwards on `[1, 2, 3, 4, 5]` and let them find 3 at the root above 5.

Lab 11 — Week 11

Draw everything on the board; this lab is won or lost on paper. Use the lab tree $T = [15, 6, 20, 3, 9, 18, 24, 7, 12, 22]$ throughout — the students' checkpoints use it, so your drawing is their answer key.

1. Build T on the board one insert at a time, with the room calling "left" or "right" at each node. Stop at 9 and ask where it goes **and whose link changes**: 6's right. Say the rule: insert stops at the parent.
2. At the REPL, show `BinarySearchTree([1, 2, 3, 4, 5]).height() → 4` and `BinarySearchTree([3, 1, 5, 2, 4]).height() → 2`. Same values, different height: the insertion order decides.
3. Draw the lecture's broken tree — 8, with 3 and 10, and 9 as the right child of 3. Ask whether it is a BST. Collect votes, then search it for 9 by hand: right at 8, never found. Write the ranges $(-\infty, 8)$, $(3, 8)$ beside the nodes.
4. Walk T four ways with the room: pre-order and post-order first, then in-order — and let them notice it came out sorted. Then level-order, with the queue drawn as a row of boxes after each dequeue (the lab's Checkpoint 6 table).
5. **The main event: delete.** On a fresh drawing of T , delete 3 (leaf), 6 (one child), 15 (root, two children: successor 18), 20 (one child). Then, on the original T , delete 9: the successor is 12, 9's **own right child**, and the "go left" loop runs zero times.
6. **Write one method live, wrongly, and fix it.** Write the two-children case of delete with `successor.parent.left = successor.right` and run `pytest tests/test_tree.py -v -k delete.test_delete_node_with_two_children passes`; `test_delete_the_root` and `test_delete_everything` fail at `assert bst.is_valid()`. Ask the room why the two-children test passed (its successor, 4, is deep; the root's, 10, is the right child). Fix it by moving node and parent down to the successor and letting the one-child code do the removal, then run again: 6 passed.

7. Warn about the filter: `-k tree` matches the file name `test_tree.py` and selects all 26. Point them at the lab's table of filters.

Lab 12 — Week 12

1. Write `[2, 4, 3, 9, 7, 8, 5, 12, 10]` on the board as an array, then ask the room to draw it as a tree using only $2i + 1$ and $2i + 2$. Check with `draw_array_as_tree` projected. Ask for the parent of index 7 and of index 8 — both 3 — and for the last parent ($9 // 2 - 1 = 3$).
2. Read `_beats` aloud and show `MaxHeap`: one line different. Say the rule of the week: no `<` or `>` between two values anywhere else in the class.
3. Push 1 into the board heap with the room calling out each parent index and each swap: $9 \rightarrow 4 \rightarrow 1 \rightarrow 0$, three swaps. Say “one comparison per level”.
4. **The main event.** Pop the board heap. Write `_sift_down` live, **wrongly**: `if left < size and self._beats(items[left], items[index]): child = left, elif right < size and self._beats(items[right], items[index]): child = right, else: return`. Run `pytest tests/test_heap.py -q` once `heapify` exists (or paste the reference `heapify` for the demo): 6 fail, including `test_invariant_holds_after_every_pop` with `assert False`. On the board, pop `[2, 4, 3, 9, 7, 8, 5, 12, 10]`: 10 goes to the root, 4 beats it and moves up — above 3. Ask the room what the next pop returns (4, not 3). Then fix it live: `best = index, compare left with items[best], then right with items[best], stop if best == index`. Run again: 28 pass.
5. Heapify `[9, 4, 7, 1, 8, 2, 6, 3, 5]` on the board from index 3 back to 0 (the lecture's trace). Ask why the loop cannot run forwards; if nobody answers, sift index 0 first on `[5, 4, 3, 2, 1]` and let them find 1 under 3.
6. Show the `-k` pitfall: `pytest tests/test_heap.py -k heap --collect-only -q` collects all 28, because the file name contains “heap”. The lab's filters avoid the word.

Lab 13 — Week 13

1. At the REPL: `hash(42), hash(-1), hash(3.0) == hash(3), hash([1])`. Then, from the terminal, `python -c "print(hash('dsa'))"` twice — the room sees two different numbers. Say the rule once: **integer hashes are the same every run, string hashes are not**, which is why every trace this week uses integers. Show `$env:PYTHONHASHSEED = "0"` and remove it again.
2. On the board, a row of 8 buckets. Put 3, 11, 6, 19, 14 with the room calling out `key % 8` (Checkpoint 2): chains $19 \rightarrow 11 \rightarrow 3$ and $14 \rightarrow 6$. Ask why the new key goes on the **front**, and what `put` must do before it pushes (look for the key, or it duplicates).
3. Ask what happens to 10 when 8 buckets become 16. Let the room say “it moves to 10”, then ask what a bucket-by-bucket copy would do to get (10). Do not write `_resize`.
4. **The main event.** Draw 8 slots and put 3, 11, 19 (all home 3): slots 3, 4,
 5. Delete 11 by **erasing** it, and ask the room to run `get(19)`: 3, then 4 is empty — “not here”. Then redraw slot 4 as `†`, and run it again: skip, 5, found. Write the three-row rule on the board: None stops a search; a tombstone does not; put may reuse a tombstone only after probing on to None.

5. **Write one method live, wrongly, and fix it: `OpenAddressingHashMap.delete` with `self._keys[i] = None`.** With a working table projected, make the change and ask the room which of the 17 tests will fail. Collect guesses, then run `pytest tests/test_hashmap.py -v`: exactly **one** fails, `test_open_addressing_delete_preserves_p` with `KeyError`: 8. Ask why `test_delete[OpenAddressingHashMap]` still passes (it deletes the only key: no probe chain runs through the slot). Put `TOMBSTONE` back.
6. Show the `-k` trap: `-k OpenAddressing` collects 8 tests and **misses** the tombstone test; `-k "OpenAddressing or open_addressing"` collects all 9.

Lab 14 — Week 14

The lab stands on four earlier structures. Spend the first ten minutes on Part 0.2: a `CircularQueue` that cannot grow, or a `ChainingHashMap` whose `in` fails on a `None` value, surfaces here as a baffling graph failure.

1. Draw the sample graph (A–B, A–C, B–C, B–D) on the board, then its adjacency list and its matrix beside it. Ask the room to count cells: 12 against 16. Then ask what the counts are for 100,000 junctions of a road map with three roads each (about 400,000 against 10^{10}). That is the whole argument for the list.
2. At the REPL, with the reference (`python tools/with_solutions.py` is not interactive, so run `python` and `import solutions; solutions.activate()` first): build the sample graph, then call `g.add_node("A")` and show `g.neighbours("A")` is unchanged. Ask what a version without the `if` would print (Checkpoint 2). Do not show the code of `add_node`.
3. On the board, BFS from A on the lecture's seven-node graph G (A–B, A–C, B–D, C–D, C–E, D–F, E–F, F–G), with the room calling out the queue after each take: B C, C D, D E, E F, F, G, empty. Stop at "take C" and ask why D is not added again — it is already **marked**, because it was marked when B enqueued it.
4. **The main event.** Same graph, same start, with a stack: the room calls out pops and pushes, pushing neighbours in reverse and marking on **pop**. C is pushed twice (by A and by D) and D's copy comes off first: A B D C E F G, the recursive order. Then do it again marking on **push**: A B D F E G C. Both are depth-first; only the first matches the recursion.
5. **Write one method live, wrongly, and fix it.** Write `dfs_iterative` marking on push:

```
visited.put(start, True)
stack.push(start)
while not stack.is_empty():
    node = stack.pop()
    order.append(node)
    neighbours = graph.neighbours(node)
    for i in range(len(neighbours) - 1, -1, -1):
        if neighbours[i] not in visited:
```

```
visited.put(neighbours[i], True)
stack.push(neighbours[i])
```

Run `pytest tests/test_graph.py -v -k "dfs or search"`: 8 pass, and only `test_dfs_and_dfs_iterative_agree` fails, on both classes, with `assert ['A', 'B', 'C', 'D'] == ['A', 'B', 'D', 'C']`. Ask which of the two lists is the recursive one, and why C comes last in the other (A marked it when it pushed it, so B could not push it again). Fix: mark on pop, skip a node popped a second time. 10 passed.

- Two minutes on cycles: the tree A-B, A-C, B-D, B-E on the board, “at B, neighbour A is visited — cycle?”. Then the diamond $A \rightarrow B, A \rightarrow C, B \rightarrow D, C \rightarrow D$ with the room colouring grey and black. Say “visited is not the same as on the path” and stop; Part 6 is homework.

Lab 15 — Week 15

The last lab of the course. Keep the demonstration to 25 minutes: the students need the session for the parser, and the last 15 minutes are the course wrap-up (step 7).

- At the REPL, from `dsa.translation import tokenize` (use the reference: start the REPL after `import solutions; solutions.activate()`) and show `tokenize("12 * (3.5+4)")`. Ask how many tokens before pressing Enter (7, not 12). Then `tokenize("3 + + 4")`: the lexer accepts it, and the room should say which stage will not.
- On the board, write the three grammar rules and parse `3 + 4 * 2` **by hand**, splitting at `+` first, then at `*`. Draw the tree. Ask: where is the precedence table? There is none — `*` is deeper because it lives in term.
- Write the lecture’s `expr` on the board — the only method the lecture shows in full — and trace it on `1 - 2 - 3 + 4` with the room, drawing the tree after each turn of the loop. Point at `BinOp(op, node, self.term())`: the old tree goes **left**.
- The main event: one character of grammar, one wrong answer.** With the reference parser projected, replace the body of `expr` with the right-recursive version (if instead of while, and `self.expr()` as the right operand). Ask the room what `1 - 2 - 3` gives, and how many of the 55 tests will fail. Then run `pytest tests/test_translation.py -q: exactly3 fail — test_subtraction_is_left_associative, test_calculate[1 - 2 - 3--4.0]` (`assert 2.0 == -4.0 ± 4.0e-06`) and `test_both_routes_agree`. Ask why `8 / 4 / 2` still passes (it is in term, which still loops).
- Then the left-recursive version, first line `left = self.expr()`: run with `-x` and show `RecursionError` on the single number `"3"`. Say the rule from Lecture 03: a recursion must make progress; this one consumes nothing. Restore the loop.
- Show `draw_tree(to_edges(Parser(tokenize("(3 + 4) * (5 - 2)")).parse()))` in the notebook, and read its post-order aloud: `3 4 + 5 2 - *` — the postfix of Lab 06.
- Wrap-up (last 15 minutes).** Run `pytest -q` on the reference (`pytest --solutions`

-q) to show the whole course passing, then walk the lab's Part 10: the exam format (30 MCQ + 30 written), the pass rule ($\geq 60\%$ overall and $\geq 30\%$ of the final), and the preparation plan. Remind them that attendance is recorded today too.

Write one method live, wrongly, and fix it. Write `factor` with the parenthesis case checking the `)` but **not** consuming it (no `self.advance()` after the check). Run the parser filter, `pytest tests/test_translation.py -k "parse or preced or assoc or unary or nested"`: 12 pass, and `test_parentheses_override_precedence` and `test_nested_parentheses` fail with `ValueError: unexpected ')'` after the expression. Ask the room which method raised it (parse, finding a token left over) and why the error appears so far from the bug. Add `self.advance()`: 14 passed. The lesson: the check and the consumption are two separate steps, and an error message names where the damage was **noticed**, not where it was **done**.

4. Exercise notes

For each exercise: what it tests, and the mistakes you will see most. The tests are the definition of correct — when in doubt, read `tests/test_labNN.py`.

Lab 01

Exercise	What it is really testing	Common mistakes
<code>seconds_to_hms</code>	<code>//</code> and <code>%</code> twice; <code>:02d</code>	<code>seconds / 3600</code> (a float); padding the hours; forgetting that minutes are <code>seconds % 3600 // 60</code> , not <code>seconds // 60</code>
<code>celsius_to_fahrenheit</code>	float arithmetic, <code>round(x, 1)</code>	printing instead of returning; <code>9 // 5</code> (integer division gives 1)
<code>split_evenly</code>	returning two values	returning a list <code>[a, b]</code> — the test compares with a tuple, and <code>[33, 1] != (33, 1)</code>
<code>initials</code>	<code>split()</code> with no argument; building a string in a loop	<code>split(' ')</code> , which produces empty strings for repeated spaces and then <code>''[0]</code> raises <code>IndexError</code>
<code>is_palindrome_word</code>	<code>strip</code> , <code>lower</code> , <code>[::-1]</code>	returning <code>'True'</code> (a string) or 1; the test uses <code>is True</code>
<code>mask_email</code>	<code>index('@')</code> , slicing, <code>'*' * n</code>	off-by-one in the number of stars; forgetting the short-name case

Exercise	What it is really testing	Common mistakes
middle	slice arithmetic	a chain of ifs that breaks for length 0 or 1; the one-slice answer exists
rotate_left	%, two slices, not mutating	$k \% 0$ on the empty list; mutating the input with pop/append — a test checks the original is unchanged
fib_list	the tutorial loop, collecting into a list	an off-by-one: $n = 1$ must give $[0]$
collatz_steps	a while with an if inside	counting the starting number as a step; $n / 2$ (a float — it still terminates, but the habit is wrong)

Rule for Lab 01: no `for`, no `import`. A student who uses `for` from prior knowledge has not failed — but ask them to rewrite one function with `while`, because the point of the lab is the while-loop mechanics they will need for binary search in Week 8.

Lab 02

Exercise	What it is really testing	Common mistakes
classify_triangle	validation before classification; raise <code>ValueError</code>	$\geq$ versus $>$ in the inequality (1, 2, 3 is degenerate and must be rejected); checking only one of the three inequalities; returning an error string
course_result	order of conditions	checking the total before attendance; <code>final < 30</code> (it is 30% of 60 , i.e. 18); not validating ranges
fizzbuzz	<code>for</code> , %	testing % 3 before % 15, so 15 becomes "Fizz"; returning ints instead of strings
is_prime	early return; the $\sqrt{n}$ bound	<code>range(2, int(n ** 0.5))</code> — misses the square root itself, so 9 and 25 come out prime; forgetting $n < 2$
primes_below	reuse	re-implementing the primality test inline

Exercise	What it is really testing	Common mistakes
first_repeated	nested loop, early exit	returning the first value that <i>has</i> a duplicate somewhere later, instead of the value whose <i>repeat comes first</i> : for [2, 5, 5, 2] that gives 2, but the answer is 5. Ask them to trace it by hand
count_pairs	counting, then a closed form	the formula $n * (n - 1) / 2$ returns a float ; they need <code>//</code> . Both loop and formula pass — ask which is $O(1)$
calculator	match, capture patterns, <code>*rest</code>	case ["sum", *rest] accepts "sum" with no numbers — the test requires <code>ValueError</code> ; a bare <code>except:</code> that also swallows their own bugs
parse_int	<code>try/except ValueError</code>	<code>text.isdigit()</code> checks (fails on " -7 "); catching <code>Exception</code>
stats	<code>*args</code>	starting min/max at 0; using min/max (not allowed); integer division for the mean
append_to	the mutable default	writing <code>target=[]</code> — the second test call fails, which is exactly the lesson
make_multiplier	returning a function	returning <code>k * x</code> with <code>x</code> undefined; calling the lambda instead of returning it
apply_n	passing a function	calling <code>f</code> once; recursion without a base case for <code>n = 0</code>

Lab 03

Exercise	What it is really testing	Common mistakes
unique_in_order	set for membership, list for order	if <code>v</code> not in <code>result</code> — $O(n^2)$; the 400,000-item test takes minutes. Let them see it hang, then ask why. <code>list(set(values))</code> loses the order
word_frequencies	<code>split</code> , <code>strip(chars)</code> , <code>get</code>	<code>replace</code> to delete punctuation (also removes it from inside words); not skipping words that become empty
top_k	sorting by a tuple key	sorting by count only (ties come out in dictionary order); <code>reverse=True</code> on a tuple key reverses the alphabetical tie-break too
transpose	nested comprehension	the order of the two fors; the empty matrix (<code>matrix[0]</code> raises)
invert	grouping	overwriting instead of appending (<code>out[v] = k</code>); forgetting to sort the key lists
common_elements	<code>&</code>	returning a set instead of a sorted list
group_by_length	<code>setdefault</code> or <code>get</code>	sorting the words (the test expects input order)
Bag countdown	a class with state; <code>dunders</code> <code>yield</code>	see below building and returning a list — the test checks it is a generator
chunks	a generator with <code>range(0, n, size)</code>	no explicit size check: <code>range(0, n, 0)</code> happens to raise <code>ValueError</code> for size 0, but a negative size silently yields nothing — the test checks both; returning a list

The Bag class is the most important exercise of the three labs, because it is the shape of every `dsa/` class. Check, in this order:

1. The counts live in a **dictionary created in `__init__`**, not a class variable. (The test that two bags do not share state catches the class-variable version.)

2. `__len__` is $O(1)$: a running total, not `sum(self._counts.values())`. The tests cannot detect the $O(k)$ version — so ask the student what `len` costs. This is a “know-why” question, the kind the exam asks.
3. `remove` deletes the key when a count reaches zero, and raises `KeyError` — not `ValueError`, not `print`.
4. `__eq__` compares the dictionaries, and returns `NotImplemented` for non-bags.
5. `__repr__` is `Bag([...])` with items **sorted**, repeats included.

Lab 04

Method	What it is really testing	Common mistakes
<code>_resize</code>	allocate, copy size elements, switch; keeping <code>_capacity</code> and <code>resize_count</code> in step	forgetting to assign <code>self._block = block</code> (the writes go to the old block); not updating <code>_capacity</code> ; no <code>resize_count += 1</code> (<code>resize_count</code> never went up – increment it in <code>_resize</code>); copying into a Python list first (storage rule)
<code>append</code>	resize before writing; multiplying by <code>self.growth</code>	writing first, then checking (Array index 1 out of range for length 1); <code>self._capacity + self.growth</code> (only the <code>resize-count</code> test fails: 500 reallocations); hard-coding <code>* 2</code> and ignoring growth
<code>__getitem__</code>	bounds against size, negative translation by size	comparing with <code>_capacity</code> (<code>[1, 2, 3][3]</code> returns <code>None</code> : DID NOT RAISE); <code>index += self._capacity</code> ; letting the Array raise for negatives
<code>__setitem__</code>	same shape as <code>__getitem__</code>	no bounds check at all — writes an unused slot silently (<code>test_setitem_out_of_range_raises</code> DID NOT RAISE)

Method	What it is really testing	Common mistakes
<code>insert_at</code>	<code>IndexError</code> unless <code>0 <= index <= size</code> , checked before anything moves; resize when full; shift right from the end	shifting from the front (<code>[1, 99, 2, 2]</code>); forgetting <code>_size += 1</code> ; no resize when full (<code>test_insert_at_grows_when_full</code> : Array index 1 out of range for length 1); clamping like <code>list.insert</code> or translating negatives (DID NOT RAISE); no index check at all (<code>test_insert_at_out_of_range_raises</code> and <code>[7]</code> : DID NOT RAISE; <code>[-1]</code> still passes, because the Array raises mid-shift, after corrupting the array to <code>[1, 1, 2, 3, 4]</code>)
<code>pop</code>	empty check, negative translation, shift left from the front, clear the freed slot	no translation of the default <code>-1</code> ; shifting the wrong way (<code>assert [1] == [2]</code>); not clearing the slot (<code>test_pop_clears_the_freed_slot</code> : <code>assert 'b' is None</code>); <code>loop range(index, self._size)</code> — crashes only on a full array, and no test catches it

The method that matters most is `append`, and in particular the choice of capacity * growth. It is three lines, and the only one of them that matters for the exam is the multiplication: ask every student what would change with `+ 2`, and expect “it still works, but resizes every other `append`, so n appends cost $\Theta(n^2)$ ”. Students who pass all 19 tests often cannot say this. Also ask to see *where* `insert_at` checks its index: it must be the first thing it does. A check placed after the shifting loop fails `test_insert_at_out_of_range_raises [-1]` (“check the index BEFORE shifting anything”): the course Array raises for index `-1` mid-shift, after the values have already moved.

Lab 05

Method	What it is really testing	Common mistakes
push_front	the order of two reference assignments; <code>_size</code>	setting head first — a self-loop, and the test hangs with no output; forgetting <code>_size += 1</code> (assert 0 == 3)
append	the empty list; stopping on the last node	no empty case — <code>AttributeError: 'NoneType' object has no attribute 'next'</code> in 19 of the 24 tests, because <code>__init__</code> calls <code>append</code> ; while node is not None walks off the end (the message ends “and no dict for setting new attributes”); <code>_size</code> in one branch only
insert_at	stopping at index - 1; reusing <code>push_front</code> ; the <code><=</code> bound	walking index steps (the new node lands one place right); no validation (-1 quietly inserts after the head: DID NOT RAISE); <code>_size</code> added twice when index 0 calls <code>push_front</code>
pop_front	the empty check first	reading <code>self.head.value</code> on an empty list (<code>AttributeError</code> , not <code>IndexError</code>)
remove	the head special case; looking one node ahead	no head branch (assert False is True on <code>remove(1)</code>); forgetting <code>_size -= 1</code> in one or both branches — only <code>test_remove_updates_the_length</code> catches it (assert 2 == 1 or assert 3 == 1)
find	the traversal loop that visits every node	while <code>node.next</code> is not None never checks the last node (assert -1 == 2) and crashes on an empty list

Method	What it is really testing	Common mistakes
<code>__getitem__</code>	range check before walking	no check: <code>AttributeError</code> instead of <code>IndexError</code> ; then calling <code>self[i]</code> in a loop inside their own methods — $O(n^2)$ that no test detects
<code>reverse</code>	holding an invariant; saving <code>next</code> first	stepping with <code>node = node.next</code> after re-linking (list becomes <code>[1]</code>); forgetting <code>self.head = prev</code> (also <code>[1]</code> , with <code>len</code> still 4); rebuilding with new nodes, or writing the values back into the old nodes (either way only the relinks test fails — the second with “the old last node must become the head: relink, do not copy values”)

reverse is the method that matters most. It is the one that cannot be written by pattern-matching the lecture’s code: the student must hold the invariant — `prev` heads the reversed part, `node` heads the rest — and see why the next node must be saved before the link is turned. Insist on the paper trace (the table in Part 7) before any code; a student who has filled it in writes the loop in a minute, and one who has not will try orders at random until the tests pass. Watch for shortcuts through a Python `list`: building new nodes, or writing the reversed values back into the existing nodes. The relinks test now catches both — the second by checking that the old last node object became the head — but a student who hit that message should be able to say why copying values is not reversing a linked list. Ask them to point at the line that turns a link round.

Lab 06

Method	What it is really testing	Common mistakes
<code>Stack.push</code>	the top at the end of the <code>DynamicArray</code>	<code>insert_at(0, value)</code> — passes every test, $O(n)$; a Python <code>list</code> as <code>self._items</code> (storage rule)

Method	What it is really testing	Common mistakes
<code>Stack.pop</code>	same end as push; the empty contract	<code>self._items.pop(0)</code> (a queue: <code>assert 1 == 3</code>); returning <code>None</code> when empty (DID NOT RAISE <code>IndexError</code>)
<code>Stack.peak</code>	reading without changing	calling <code>pop</code> (length drops: <code>assert 1 == 2</code>); <code>self._items[0]</code> (the bottom); <code>self._items[len(self._items)]</code> (<code>IndexError: 2</code>)
<code>is_balanced</code>	all three checks	no partner check (<code>(([])</code> gives <code>True</code>); no non-empty check (<code>()</code> raises <code>IndexError: pop from empty stack</code>); return <code>True</code> at the end (<code>()</code> gives <code>True</code>); a Python list as the stack
<code>evaluate_postfix</code>	operand order; malformed input	swapped operands (<code>assert -2 == 2</code> ; the exact-division and division-by-zero tests fail too); no count check before popping (<code>IndexError</code> instead of <code>ValueError</code>); no “exactly one left” check; <code>//</code> instead of <code>/</code> (<code>assert 3 == 3.5</code>)
<code>infix_to_postfix</code>	equal precedence pops; (as a barrier	<code>></code> instead of <code>>=</code> — gives <code>8 3 2 - -</code> , caught only by <code>test_infix_to_postfix_is_left_associative</code> asking the precedence table about (<code>(KeyError: '('</code>); (not discarded (it appears in the output); no flush at the end

infix_to_postfix matters most. Its first test never puts two operators of equal precedence side by side, so only `test_infix_to_postfix_is_left_associative` (`8 - 3 - 2, 8 / 4 / 2`) catches the `>-for->=` bug, with `assert ['8', '3', '2', '-', '-', '-'] == ['8', '3', '-', '2', '-']`. When a student sees that message, ask them to evaluate both lists by hand (3 against 7) before they touch the code. Neither test has two pairs of parentheses, so also ask for `'( 1 + 2 ) * ( 3 - 4 ) / 5'.split()` in front

of you: `1 2 + 3 4 - * 5 /`, which their `evaluate_postfix` makes -0.6. Also watch for `test_postfix_pairs_with_the_stack_exercise` in `tests/test_translation.py`: it needs the Week 15 `tokenize`, so students who run `-k postfix` see one failure that is not theirs. The lab and the lecture use `-k evaluate_postfix` (11 tests).

Lab 07

Method	What it is really testing	Common mistakes
<code>SlowQueue.enqueue</code>	using the <code>DynamicArray</code> through its public methods	reaching into <code>self._items._block</code> ; adding a Python list beside it
<code>SlowQueue.dequeue</code>	which end is the front; raising on empty	<code>pop()</code> (a stack: <code>assert 3 == 1</code>); return <code>None</code> when empty (DID NOT RAISE <code>IndexError</code>); forgetting <code>return</code>
<code>CircularQueue.enqueue</code>	the tail formula with <code>%</code> ; full by size	no <code>%</code> (Array index 3 out of range for length 3 in the wraps test); full tested as <code>tail == head</code> , so the first enqueue raises “queue is full”; forgetting <code>size += 1</code>
<code>CircularQueue.dequeue</code>	wrapping head; order of read and clear	<code>self._head += 1</code> without <code>%</code> — passes the wraps test, fails only the interleaved one with Array index 4 out of range for length 4; clearing the slot before reading it (<code>assert None == 1</code>); no empty check
growth (challenge)	copying in queue order; resetting three fields	the naive slot-i-to-slot-i copy; forgetting <code>head = 0</code> or the new capacity; computing the slot before growing; <code>2 * 0</code> for <code>CircularQueue(0)</code>

`CircularQueue.dequeue` matters most, because its classic bug hides: a head that never wraps still passes three of the four `CircularQueue` tests, including the one named “wraps around” — that test dequeues once and reads the queue through the given `__iter__`, which does its own `%`. Only `test_circular_queue_interleaved_operations` reaches the fifth dequeue, at `block[4]`. When a student says “the wrap test passes, so my wrap

is right”, ask them to trace the interleaved test by hand. For the growth challenge the provided tests prove nothing (they never overfill a ring): check that the student wrote their own test on a **wrapped** ring (Part 6.4 of the lab), not only on a ring with head 0, where the naive copy happens to work.

Lab 08

Method	What it is really testing	Common mistakes
<code>linear_search</code>	the loop shape; return <code>-1</code> after the loop; the first match	return <code>-1</code> inside the loop (assert <code>-1 == 2</code> , and assert <code>-1 == 1</code> in the first-match test); returning the last match (assert <code>4 == 1</code> in <code>test_linear_search_returns_the_first_match</code>); <code>lo < hi</code> (4 tests fail); <code>lo = mid / hi = mid</code> (a test hangs); / for // (TypeError: list indices must be integers or slices, not float); no final return <code>-1</code> (assert <code>None == -1</code> , 6 tests); <code>hi = len(values)</code> (IndexError, 4 tests, including the outside targets 10 and 99)
<code>binary_search</code>	an inclusive range kept consistent	slicing (assert <code>1 == 4</code> : the index is into the slice); a call without return (assert <code>None == ...</code> , 9 of 10 fail); if not <code>hi</code> : instead of if <code>hi is None</code> : — only the random test fails, with <code>RecursionError</code>
<code>binary_search_recursive</code>	recursion with <code>lo/hi</code> parameters; a default that depends on another argument	

Method	What it is really testing	Common mistakes
<code>lower_bound / upper_bound</code>	a half-open range, and never stopping early	inclusive <code>hi = len - 1</code> (<code>assert 4 == 5: len(values) unreachable</code>); returning <code>-1</code> past the end; <code><</code> in <code>upper_bound</code> (it becomes <code>lower_bound</code> : <code>assert 1 == 4</code> , and <code>assert 0 == 4</code> in the duplicates test); <code>lo <= hi</code> with <code>hi = mid</code> hangs <code>n ** 0.5</code> (a float index: <code>TypeError</code>); walking to <code>prev + step</code> without <code>min(..., n)</code> — <code>IndexError</code> only in <code>test_target_outside_the_values</code> for 10 and 99, the targets that walk off the short last block
<code>exponential_search</code>	slot 0, doubling from 1, capping the bound	<code>bound = 0</code> (hangs: <code>0 * 2</code> is 0); no <code>bound < n</code> before <code>values[bound]</code> (<code>IndexError</code> , 5 tests); <code>hi =</code> <code>bound</code> uncapped (<code>IndexError</code> , 4 tests)
<code>interpolation_search</code>	the three traps of the lecture	no equal-ends check (<code>ZeroDivisionError</code> : integer division or modulo by zero on [42]); no out-of-range check — hangs on 4 in [1, 3, 5, 7, 9], because the guess lands left of <code>lo</code> ; <code>/</code> in the formula (<code>TypeError</code> , 6 of 10 fail)

binary_search is the one that matters most — for the lab, for the final, and for every language the students will use after this course. The tests catch the loud bugs, but check two things by reading the code. First, that the student can state what `lo` and `hi` mean (both inclusive) and derive `lo <= hi` and `mid ± 1` from that, rather than remembering them; ask them to explain why the bounds' loop uses `lo < hi` and `hi = mid` and theirs does not. The `lo < hi` bug still passes 7 of the 11 tests, so reading matters. Second, that `values[mid]` is read once per turn and that nothing slices or calls `in`, `index` or `bisect` —

the million-element test catches a linear scan, but not a stray slice in the recursive version (the slicing version fails only because it loses the offset). A student who passes all 11 and cannot trace `[3, 8]` for 8 by hand has not finished.

Lab 09

Method	What it is really testing	Common mistakes
<code>bubble_sort_steps</code>	the pass structure; fresh snapshots; the early exit	<code>range(end + 1)</code> (<code>IndexError: Array index 5 out of range for length 5, 10 of 12</code> fail); if not swapped: break inside the inner loop (<code>assert [2, 5, 1, 7, 9] == [1, 2, 5, 7, 9], 4</code> fail); yielding a in the last yield (<code>assert [Array([1, 2, 5, 7, 9]), ()] == [1, 2, 5, 7, 9]</code>); <code>swapped = False</code> before the outer loop — passes all 12 , but the early exit never fires again
<code>selection_sort_steps</code>	the minimum <i>so far</i> ; one swap per round	comparing with <code>a[i]</code> instead of <code>a[smallest]</code> (<code>assert [1, 2, 7, 5, 9] == [1, 2, 5, 7, 9], 4</code> fail); swapping inside the inner loop — passes all 12, and makes 15 swaps instead of 3 on a reversed list of 6
<code>insertion_sort_steps</code>	the order of the and; <code>></code> for stability; writing current back	<code>a[j] > current</code> and <code>j >= 0</code> (<code>IndexError: Array index -1 ..., 9 of 13</code> fail); <code>>=</code> (only the stability test fails); no <code>a[j + 1] = current</code> after the loop (<code>assert [5, 5, 5, 9, 9] == [1, 2, 5, 7, 9], 7</code> fail)

Method	What it is really testing	Common mistakes
the plain forms	reusing the generator	returning the generator itself (assert <generator object ...> == [3, 3, 3], 9 fail); a second copy of the loops — passes, but it is two implementations to keep correct; sorting the caller's list (a = values: only the mutation test fails)
counting_sort	k = max + 1; the empty list	k = max(values) (IndexError: Array index 5 out of range for length 5); no empty check (ValueError: max() iterable argument is empty); a Python list for counts (storage rule — read the code)

bubble_sort_steps matters most, because two of its bugs pass every test. A swapped flag set once outside the passes, and a selection sort that swaps inside its scan, both sort correctly; only a count shows them. Make every student run the Part 6 counts and compare with the table: 999 comparisons for bubble sort on sorted input, not 499,500. The frame tests are also weak — a generator that yields a in its comparison frames but `list(a)` in its last one passes both of them — so look at every `yield` line. And check by eye that the plain forms are one line each over the generator: the tests cannot see a duplicated algorithm.

Hints, smallest first:

- *IndexError in bubble sort.* “Which two slots does the last comparison of pass 1 read?” → “So what is the largest j?” → “`range(end)` stops where?”
- *The stability test fails.* “What does your loop do when `a[j]` equals current?” → “Should an equal value let current pass?” → “Which character decides?”
- *The frames tests fail.* “Print `type(frame[0])` for your last frame.” → “What does `list(a)` give that a does not?”
- *The count on sorted input is 499,500.* “Where do you set `swapped = False`?” → “What is its value at the start of pass 2?”
- *counting_sort raises IndexError.* “Which values can occur? How many counters is that?”

Lab 10

Method	What it is really testing	Common mistakes
<code>_merge</code>	three indices on one Array; the tie rule; the copy back	<code><</code> in place of <code><=</code> — passes all 37 , breaks stability; one leftover loop missing (TypeError: ' <code><=</code> ' not supported between instances of ' <code>int</code> ' and ' <code>NoneType</code> ', 8 tests); no copy back into a (7 fail, including “frames never change”)
<code>merge_sort_steps</code>	a recursive generator; the base case	a plain call <code>sort(lo, mid)</code> without <code>yield</code> from — 5 fail, and <code>[5, 2, 9, 1, 7]</code> comes back unchanged; base case <code>hi - lo <= 0</code> (RecursionError, 11 fail); a new scratch Array in every call (correct, but n allocations — say so)
<code>_partition</code>	Lomuto’s invariant; inclusive <code>lo..hi</code>	no final swap of the pivot into a <code>[boundary]</code> (7 fail, e.g. <code>assert [1, 5, 2, 9, 7] == ...</code>); returning <code>j</code> or <code>hi</code> instead of <code>boundary</code> ; <code>a[j] <= pivot</code> (still correct, but every value equal to the pivot moves left)
<code>_choose_pivot</code>	returning an index ; median of three by comparisons	returning the pivot value (IndexError, or silent nonsense when values are small integers); <code>sorted(...)[1]</code> — forbidden, and returns a value, not an index

Method	What it is really testing	Common mistakes
<code>quick_sort_steps</code>	swap the pivot to <code>hi</code> ; smaller side first	never swapping the chosen index to <code>hi</code> — passes all 37 , every strategy acts like "last"; if instead of while (4 fail: the larger part is never sorted); two recursive calls — passes all 37 , <code>RecursionError</code> on 1,000 sorted values with "first"
<code>_sift_down</code>	the larger child; size as the bound	comparing only the left child (6 fail); comparing the right child with <code>a[index]</code> instead of <code>a[largest]</code> — can lift the smaller child
<code>heap_sort_steps</code>	the build direction; the shrinking heap	build loop upwards, <code>range(n // 2)</code> — only 2 fail (<code>[1, 2, 3, 4, 5]</code> gives <code>[1, 2, 4, 5, 3]</code>); stop value 0 instead of -1 (the root never sifted, 5 fail); <code>_sift_down(a, 0, n)</code> in the sort loop (7 fail, output often reversed); <code>range(n - 1, 1, -1)</code> (<code>[2, 1]</code> stays unsorted)

Three bugs pass every test, and all three are in quicksort or the merge. The `<` in `_merge`, the pivot that is chosen but never swapped to `hi`, and the two-recursive-call quicksort. Read the student's `_merge` for `<=`, read `quick_sort_steps` for the swap line and the while, and ask for the Part 6 counts: if "median3" on sorted input costs $n(n - 1)/2$, the swap is missing. A heap sort that passes "35 of 37" has its build loop running upwards.

Graded hints for `merge_sort_steps`, smallest first:

1. "Put a `print(lo, hi)` at the top of sort. How many lines do you see for five elements?"
2. "sort contains `yield`. What does calling a generator function do?"
3. "`yield from sort(lo, mid)` — run it, and pass its frames up."

Graded hints for `quick_sort_steps`:

1. "Trace your code on `[3, 1, 2]` with `pivot=\"first\"`. Where is the pivot when `_partition` starts?"
2. "Which part of the range does your while loop handle, and which does the call handle?"

Is the call always the smaller one?"

3. "if $q - lo < hi - q$: recurse left and set $lo = q + 1$; else recurse right and set $hi = q - 1$."

Graded hints for `heap_sort_steps`:

1. "Draw your array as a tree after the build loop. Is every parent at least as large as its children?"
2. "When you sift index 0 first, are its two subtrees heaps yet?"
3. "In the sort loop, which slots belong to the heap? What should size be?"

Lab 11

Method	What it is really testing	Common mistakes
<code>insert</code>	stopping at the parent; the empty tree; duplicates	walking until node is <code>None</code> , then <code>node = TreeNode(value)</code> — the tree never grows (<code>assert 1 == 2, AttributeError: 'NoneType' object has no attribute 'value' in the structure test</code>); no equality check (<code>assert 5 == 2</code> in the duplicates test); forgetting the empty tree
<code>contains, min, max</code>	a loop down one path; <code>ValueError</code> on empty	returning <code>False</code> after the first comparison; return <code>None</code> on an empty tree (<code>Failed: DID NOT RAISE ValueError</code>); a recursive <code>contains</code> — accepted, but it hits the recursion limit on a chain
<code>height, size</code>	recursion from a base case; the <code>-1</code> convention	<code>_height(None)</code> returning 0 (<code>assert 0 == -1, assert 4 == 3</code>); <code>1 + max(...)</code> in <code>_size</code> (<code>assert 4 == 9</code>); recursing on <code>self.root</code> instead of the node (<code>RecursionError</code>)

Method	What it is really testing	Common mistakes
<code>is_valid</code>	the invariant is about subtrees, not children	the parent-only check (<code>assert not True</code> in the broken-tree test); <code>if low</code> and <code>...</code> so that a bound of 0 is ignored; <code><</code> for <code><=</code> , which lets a duplicate of an ancestor through
the three recursive walks	one output list passed down; left before right	a new <code>out = []</code> inside the helper (<code>assert [] == [1, 3, 4, 6, ...]</code>); a list stored on the tree and returned each time (<code>in_order</code> handed out a list the tree still uses); <code>left + [v] + right</code> concatenation — passes, but $O(n^2)$ on a chain
<code>level_order</code>	a queue of nodes ; the course <code>CircularQueue</code> , which must grow	a queue that cannot grow (<code>IndexError: queue is full</code> in the wide-tree test only); right child before left; enqueueing <code>None</code> for an empty tree (<code>AttributeError: 'NoneType' object has no attribute 'value'</code>); a Python list with <code>pop(0)</code> — passes every test
<code>delete</code>	keeping the parent; the root; the successor as the right child	no root case (<code>AttributeError: 'NoneType' object has no attribute 'left'</code> in <code>test_delete_everything</code> only); always <code>successor_parent.left</code> (root and delete-everything fail at <code>is_valid</code> , the two-children test passes); copying the successor but never removing it (three tests fail at <code>is_valid</code>); crashing on a missing value

delete matters most, and its tests pass for the wrong reasons more often than any other method's. The root case with the parent None is reached only when the root has **at most one** child, which in the sample tree happens late in `test_delete_everything`; and the successor-is-the-right-child case appears only when the root 8 is deleted. Ask every student to trace `delete(9)` on the original T by hand before you accept their delete. For `level_order`, the tests cannot see a Python list used as the queue: read the code. For `is_valid`, ask for the ranges on a drawing, not the code.

Hints, smallest first:

- *insert never grows the tree.* "Which variable holds the parent when your loop ends?" → "Look at the child **before** you step onto it." → show the three-line look-ahead on the board, without the rest.
- *is_valid passes the valid trees but not the broken one.* "Which ancestors does 9 have to beat?" → "What range does a node in 8's left subtree live in?" → "Give the helper two more parameters."
- *level_order is out of order.* "What does your queue hold after the root is dequeued?" → "Children in which order?" → draw the queue boxes.
- *delete of the root fails.* "Which node is the successor of 8? How many times did your loop go left?" → "So whose link must change — left or right?" → "What if, after copying the value, you just deleted the successor with your one-child code?"

Lab 12

Method	What it is really testing	Common mistakes
<code>is_valid</code>	one check per edge, from index 1; the check the other tests rely on	checking only the root's two children — passes all 28 tests ; starting at 0, where <code>(0 - 1) // 2</code> is -1 and the root is compared with the last leaf (valid heaps rejected); <code><</code> in place of <code>_beats</code> (5 [MaxHeap] failures)
<code>peek</code>	<code>IndexError</code> when empty	returning None (DID NOT RAISE <code>IndexError</code> in <code>test_empty_heap</code>)
<code>_sift_up</code> , <code>push</code>	the parent formula; the loop condition; argument order in <code>_beats</code>	<code>index // 2</code> (only <code>test_invariant_holds_after_every_fails</code>); while <code>index >= 0</code> (<code>assert 3 == 1</code> : the root swaps with a leaf); <code><</code> in place of <code>_beats</code> (<code>assert 1 == 9</code> in the [MaxHeap] peek test)

Method	What it is really testing	Common mistakes
<code>_sift_down</code> , <code>pop</code>	the better child; separate bounds for each child; the one-item heap; saving the root first	first-child-that-beats (6 fail in the full file, <code>assert False from is_valid</code>); <code>left < size</code> used for the right child (<code>IndexError: 2</code> or <code>6</code> from the <code>DynamicArray</code> , 10 tests); no one-item case (<code>IndexError: 0</code> from the <code>DynamicArray</code> , 9 tests); returning <code>items[0]</code> after the sift (<code>assert 21 == 14</code> in the interleaved test); one swap and no loop (4 fail)
<code>heapify</code>	direction and bounds of one range	stop at 0 instead of <code>-1</code> — the root is never sifted (5 fail, <code>assert 5 == 1</code>); forwards (5 fail); starting at <code>n // 2 - 2</code> (4 fail); a forward loop of <code>_sift_up</code> — passes all 28 , but is $O(n \log n)$
<code>PriorityQueue</code>	building on the heap; tuple comparison; the counter	(<code>priority, item</code>) pairs (<code>TypeError: '<' not supported between instances of 'dict' and 'dict'</code>); a counter that is never incremented (only the unorderable test fails); returning the triple (<code>assert (1, 1, 'fix the build') == 'fix the build'</code>); re-implementing sifting inside the priority queue

Two correct-looking methods pass every test and are still wrong. An `is_valid` that checks only the root's children passes all 28 — so read every student's `is_valid`: if it does not loop over all children, their other methods have never really been checked. And a `heapify` written as a forward loop of `_sift_up` passes all 28 while being $O(n \log n)$: ask to see the range, and ask for the Part 8 counts (2.00 per element at $n = 16,384$ on descending input for a real `heapify`; 12.00 for the push version).

Graded hints for `_sift_down`, smallest first:

1. "Draw [10, 4, 3, 9, 7] and sift the root down by hand. Which child did you pick, and why?"
2. "Your code decides on the left child before it has looked at the right one. What if both beat the item?"
3. "Keep a variable `best` that starts at `index`. Compare each child with `items[best]`, not with `items[index]`."
4. "Before reading `items[right]`, check that `right` exists — separately from `left`."

Graded hints for `pop`:

1. "What does your `pop` do when the heap has exactly one item? Trace it."
2. "After `self._items.pop()`, how many items are left? What is at index 0?"
3. "Save the root in a variable **before** you overwrite index 0."

Graded hints for `heapify`:

1. "Which is the last index that has a child?"
2. "Print `list(range(3, 0, -1))`. Is 0 in it?"
3. "When you sift down index `i`, what must already be true of its two subtrees?"

Lab 13

Method	What it is really testing	Common mistakes
<code>ChainingHashMap.get</code>	walking a chain; the <code>_MISSING</code> sentinel	<code>return None</code> instead of <code>raise KeyError</code> (DID NOT RAISE <code>KeyError</code> in <code>test_get_missing_key</code>); <code>if default:</code> or <code>if default is None:</code> as the test, so <code>get(k, None)</code> raises; comparing with <code>is</code>
<code>ChainingHashMap.put</code>	overwrite-or-push; the resize trigger	no "already there?" walk (<code>assert 2 == 1</code> in <code>test_put_overwrites_without_growing</code>); <code>Entry(key, value)</code> without the old head, which drops colliding keys; resizing with <code>>=</code> (harmless) or never (<code>assert 62.5 <= (0.75 + 0.05)</code>)

Method	What it is really testing	Common mistakes
<code>ChainingHashMap.delete</code>	Week 5's unlink, head as the special case	forgetting <code>size -= 1</code> (<code>assert 1 == 0</code>); <code>self._buckets[i] = entry.next</code> for every match — passes every test and silently drops the entries in front of a mid-chain match
<code>ChainingHashMap._resize</code>	rehashing; order of pointer updates	copying <code>old[i]</code> to <code>new[i]</code> (<code>KeyError: 'key0'</code> — the key varies); <code>entry = entry.next</code> after relinking (same <code>KeyError</code> : the rest of the chain is lost); computing indices before swapping in the new Array
<code>OpenAddressingHashMap._probe</code>	probe generator; the wrap	no % capacity (<code>IndexError: Array index 8 out of range for length 8</code> — or 16, 64, 128, 1024, in a different test each run , because the string keys move); while True: with no bound
<code>OpenAddressingHashMap.put</code> <code>/ get</code>	None stops, TOMBSTONE does not	get treating <code>†</code> as None (<code>KeyError: 8</code> in the tombstone test); put inserting at the first <code>†</code> without probing on — no test fails, duplicates appear; forgetting to count tombstones in the load check; not decrementing <code>_tombstones</code> on reuse
<code>OpenAddressingHashMap.delete</code>	the tombstone	writing None (<code>KeyError: 8</code>); forgetting <code>_values[i] = None</code> or the two counters

Method	What it is really testing	Common mistakes
<code>OpenAddressingHashMap._resize</code>	resizing the counts	not resetting <code>_size</code> (assert <code>1175 == 500</code> , the number varies); copying tombstones across (no test fails); always resizing to the same capacity (TypeError: 'NoneType' object cannot be interpreted as an integer in two tests: the table filled up)

`OpenAddressingHashMap.delete` — and what `get` does with its result — is the one that matters most, because the tests guard it with a single test and the student who “fixes” that test by trial and error has learnt nothing. Ask for the three-row rule (what `get`, `put` and `_resize` each do with a tombstone) before the check-off. The second thing to check by reading, because no test sees it, is the chaining delete on the **middle** of a chain: point at the line that unlinks and ask what happens to the entries in front of the match. Part 3.3 of the lab gives them a short check; make sure they ran it.

Hints, small to large, for a student stuck on a `KeyError` after resizing: (1) “Print `len(m)` just before the failing `get`. Is the key lost, or just not where you look?” (2) “What is `10 % 8`? What is `10 % 16`? Which one does your `get` use after the `resize`?” (3) “Walk your `_resize` on paper with one chain of two entries: after the first relink, what is `entry.next`?”

For the moving `IndexError` of a `_probe` without `%`: tell the student to set `$env:PYTHONHASHSEED = "0"`, rerun, and read the traceback. When the failure stops moving, the student will usually find the missing `%` alone.

Lab 14

Method	What it is really testing	Common mistakes
<code>Graph.add_node / add_edge</code>	not overwriting an existing entry; both directions when undirected	no <code>if node not in self._adjacent</code> — 5 of the 11 structure tests fail, e.g. <code>assert ['C'] == ['B', 'C']</code> in <code>test_neighbours</code> , and <code>nodes()</code> lists a node twice; forgetting the reverse <code>append(assert [] == ['B'], assert 2 == 3)</code> ; appending a loop twice

Method	What it is really testing	Common mistakes
neighbours, edges, degree	returning a list; KeyError; each undirected edge once	returning the DynamicArray itself (assert DynamicArray([]) == []); get(node, DynamicArray()) (DID NOT RAISE KeyError); both directions in edges() (assert 8 == 4); a Python set of seen edges (storage rule — the tests pass)
MatrixGraph	growing a 2-D structure by a row and a column	a new row but no new column: 10 of 11 fail with an IndexError from their DynamicArray; no mirror cell (assert False in the both-ways test); full matrix in edges() (assert 8 == 4)
bfs	the queue; marking on enqueue	mark on dequeue only: assert 5 == 4 in test_bfs_visits_each_node_once, and nothing else fails; no visited check: the run hangs; start not marked: A reappears, ['A', 'B', 'D', 'A', 'C']; a CircularQueue that raises when full
shortest_path_unweighted	the parent map as the visited set; walking back	no reverse (assert ['D', 'B', 'A'] == ['A', 'B', 'D']); start missing from parent: KeyError: 'A' for the path to itself and the other tests hang; a __contains__ that treats a stored None as absent (the same symptoms)

Method	What it is really testing	Common mistakes
dfs / dfs_iterative	the call stack versus their own stack	mark on push (only the agreement test fails); neighbours pushed in order (assert ['A', 'B', 'C', 'D'] == ['A', 'C', 'B', 'D']); recursion inside dfs_iterative
connected_components	one search per unseen node	no seen check: assert 4 == 1 in test_one_component_when_connected
has_cycle	parent check against three colours	no parent check (assert not True for the path A-B-C and for the tree); "visited" instead of grey, or never colouring black — both fail the DAG $A \rightarrow B, A \rightarrow C, B \rightarrow C$ and the diamond
topological_sort	Kahn's in-degrees; the length check	no length check: assert [] is None for the three-cycle; computing in-degrees with a Python dict (storage rule)

The two searches are the ones that matter — for the final and for every later course that touches a graph. The tests catch the loud bugs, but check two things by reading. First, that the student can say **when** a vertex is marked in each search and why: on enqueue in BFS (or it is queued twice), on pop in the iterative DFS (or the order is not the recursive one). A student who passes -k "dfs or search" by copying the lecture's pseudocode and cannot trace the stack on the sample graph by hand has not finished. Second, the storage rule: `visited = set()` or `in_degree = {}` passes every test. Ask them to point at their visited set.

Graded hints, smallest first, for a stuck student:

- **add_node / edges:** "What does put do to a key that is already there?" → "Draw B's list before and after `add_edge("B", "C")`." → "When you reach B in `edges()`, which of its neighbours have you already reported from the other end?"
- **bfs:** "When does D get marked?" → "Draw the queue after B is taken: is C in it, and is C marked?" → "Mark the moment you enqueue."
- **dfs_iterative:** "Which neighbour does the recursion go to first?" → "Which one comes off a stack first?" → "What must happen when a node is popped a second time?"
- **has_cycle:** "In an undirected graph, how many times is the edge A-B stored?" → "In

the diamond, is D still on the path when C reaches it?"

- **topological_sort:** "On a three-cycle, which node has in-degree 0?" → "How do you know, at the end, that some nodes never came out?"

Lab 15

Method	What it is really testing	Common mistakes
<code>tokenize</code>	one scan; numbers kept whole; the bound before the read	each digit its own token (3 of the 13 lexer tests fail: At index 0 diff: '1' != '12'); spaces appended (5 fail); no <code>i < len(text)</code> in the inner loop (IndexError: string index out of range, 7 fail — every case ending in a number); no final else (DID NOT RAISE, the 3 rejection tests); <code>ch.isdigit()</code> , which accepts '₹' — no test catches it, but point it out if instead of while
<code>Parser.expr, term</code>	a loop; the old tree on the left; <code>expr</code> calls <code>term</code>	(ValueError: unexpected '-' after the expression on 1 - 2 - 3); right recursion (3 of 55 fail, all on 1 - 2 - 3); left recursion (RecursionError, 24 fail); <code>expr</code> calling <code>factor</code> and handling all four operators (3 + 4 * 2 parsed as (3 + 4) * 2, 3 fail)

Method	What it is really testing	Common mistakes
<code>Parser.factor</code>	three cases and an error; the recursion back to <code>expr</code>	) not consumed (2 parser tests + 3 end-to-end); no None check (TypeError: 'NoneType' object is not subscriptable on "3 +"); <code>Num(token)</code> without float (assert <code>Num('3') == Num(3.0)</code> , 19 fail); unary minus with <code>self.expr()</code> — passes all 14 parser tests , fails only <code>calculate("-5 + 2")</code> (-7.0)
<code>Parser.parse</code>	refusing what no rule used	no leftover check (DID NOT RAISE on "3 + 4 "); a missing empty check is harmless — <code>factor</code> then raises on [] anyway
<code>evaluate</code>	post-order; the operand order	right - left — passes the 4 evaluator tests (they use only + and *), fails 4 end-to-end (-5 + 2 gives 7.0); <code>node.left.value</code> without recursion (AttributeError: 'BinOp' object has no attribute 'value', 8 fail); no ValueError for ^; <code>// (10 / 4 gives 2.0)</code>
<code>calculate</code>	the pipeline	rarely wrong; students sometimes call <code>evaluate_postfix</code> here — it passes most tests and fails -5 + 2 with ValueError

Check factor and expr by reading. Two bugs pass every unit test of their own stage (unary minus with `expr`; swapped operands in `evaluate`), and a precedence **table** hidden in the parser passes everything. Ask each student to point at the line where precedence is decided; the right answer is “which method calls which”. Also check that nothing uses `eval` or `re`. Graded hints, smallest first:

- *Lexer stuck*: “What is `text[i]` when `i == len(text)`?” → “Write the loop condition with the bound first.” → show the four cases as a comment skeleton.

- *Parser wrong tree*: “Draw what your code builds for 1 - 2 - 3.” → “After the loop’s first turn, where is the old node?” → point at the lecture’s expr.
- *Parser error far from the bug*: “Which method raised it? Which token was it looking at?” → “Who should have consumed that token?” → the Part 6.2 tracing subclass.
- *evaluate wrong only end to end*: “Evaluate `BinOp( '-', Num(5.0), Num(3.0))` by hand and with your code.” → “Which child is the left operand?”

5. Checking students’ work

From the student’s DSA27 folder, with their virtual environment active:

```
git status                # nothing unexpected changed?
git diff --stat tests/    # tests must be unchanged – this must be empty
pytest tests/test_lab01.py -q  # the result you record
```

If tests were edited, restore them with `git checkout -- tests/` and run again. Record the result of the original tests.

Suggested marking, per lab (the lab portion of coursework is announced by the lecturer):

Mark	Meaning
2	All tests pass, and the student explains the line you point at
1	Most tests pass, or all pass but the explanation is weak
0	Little working, or the student cannot explain their own code

The explanation matters more than the count. A student who passes 8 of 10 and can explain every line has learnt more than one who passes 10 of 10 with code they cannot read.

Know-why questions, labs 04–15

From Week 4 the tests check behaviour; these questions check understanding. Ask one about the student’s own code at check-off.

Lab 04 — know-why questions

- “Why does `append` *resize before it writes, not after?*” — Slot size of a full block does not exist; writing first raises `IndexError`.
- “What would `resize_count` be after 1,000 appends if you wrote `+` instead of `*`? Why does it matter?” — 500 with growth 2 (100 with a step of 10); the copies become $\Theta(n^2)$ in total, so `append` is amortised $O(n)$, not $O(1)$.
- “Why do you set the freed slot to `None` in `pop`?” — Otherwise the block still refers to the removed object and the garbage collector cannot free it.
- “Why does `insert_at(4, x)` on a three-element array raise, when `insert_at(3, x)` does not?” — Valid positions are `0 .. size`; inserting at `size` appends, anything past it would leave a gap.

- “What does `pop(0)` cost, and which structure next month pays it?” — $O(n)$: every other element shifts left; `SlowQueue` in Week 7 dequeues with it.

Lab 05 — know-why questions

- “Why must `new.next = self.head` come before `self.head = new` — and why does `self.head = Node(value, self.head)` not have the problem?” — The other order makes the node point at itself and loses the list; in the one-liner the right-hand side, including the old head, is evaluated before head changes.
- “Your `remove` has a separate branch for the head. Why can't the loop handle it?” — The loop changes `prev.next`, and the head has no node before it; only `self.head` refers to it (a sentinel would remove the case).
- “Point at the line in `reverse` that would lose the list if it were missing.” — The line that saves `node.next` before `node.next = prev` overwrites it.
- “What does `len(ll)` cost, and what would break if you deleted `_size -= 1` from `remove`? Would the tests notice?” — $O(1)$ because `_size` is kept up to date; `len` would over-count after every removal, and `test_remove_updates_the_length` would fail — the only `remove` test that checks `len`.
- “What does `for i in range(len(ll)): print(ll[i])` cost, and what do you write instead?” — $O(n^2)$, since every `ll[i]` walks from the head; `for v in ll:` is $O(n)$.

Lab 06 — know-why questions

- “What does your `push` cost, and why?” — $O(1)$ amortised: it appends at the end of the `DynamicArray`; nothing shifts, and the occasional `resize` averages out.
- “Your tests pass with the top at index 0 too. Why is that version wrong?” — every `push` and `pop` shifts all n items: $O(n)$ each, $O(n^2)$ for n operations; the tests check behaviour, not cost.
- “Show me the line in `is_balanced` that makes `)` return `False`.” — the non-empty check before popping; without it the `pop` raises `IndexError`.
- “In `evaluate_postfix`, which popped value is the left operand?” — the second one popped; for `8 3 -`, 3 comes off first and the answer is `8 - 3`.
- “Why `>=` and not `>` in `shunting-yard`?” — equal precedence must leave first so that `-` and `/` are left associative: `8 - 3 - 2` is `(8 - 3) - 2`.

Lab 07 — know-why questions

- “Why does your `SlowQueue.dequeue` cost $O(n)$, and would putting the front at the end fix it?” — `pop(0)` shifts every remaining item; swapping the ends makes `enqueue` `insert_at(0, x)`, which shifts instead.
- “Where does your next `enqueue` write, and why the `%`?” — `at (head + size) % capacity`; without `%` the index runs past the end of the `Array` instead of reusing the freed slots at the front.
- “Why do you store `_size`? Could you test full with `tail == head`?” — no: `tail == head` is true both when empty and when full; the size tells them apart.

- “Why set the dequeued slot to None — no test checks it?” — otherwise the block keeps a reference to the dequeued object and Python cannot free it.
- “When your ring grows, why can’t you copy slot i to slot i ?” — once the data has wrapped, the front is not at slot 0; copy the i -th queue item from $(\text{head} + i) \% \text{old_capacity}$ to slot i , then set head to 0 and the new capacity.

Lab 08 — know-why questions

- “Your loop is `while lo <= hi`. What goes wrong with `<`?” — The one-element range is never examined, so the last position (and others) can never be found.
- “Why is `lo < hi` correct in your `lower_bound`?” — There `hi` is excluded (half-open), so `lo == hi` already means an empty range.
- “Why does your recursive version take `lo` and `hi` instead of slicing?” — A slice copies ($O(n)$ in total) and its indices are relative to the slice.
- “What is `upper_bound(v, x) - lower_bound(v, x)`, and what does it cost?” — The number of copies of `x`, in $O(\log n)$ whatever that number is.
- “When is your interpolation search slower than linear search?” — On skewed values: every guess lands near `lo`, one element per step, three reads each.

Lab 09 — know-why questions

- “Why does your generator yield `list(a)` and not `a`?” — Each frame must be a snapshot of that moment; `a` is the working Array, which keeps changing, and a consumer that keeps the frames would see only the final state.
- “Your `bubble_sort` is one line. What does it cost, and why?” — $O(n^2)$ only if the plain sort does not copy per step; with a `list(a)` per yield it is $O(n^3)$ (the snapshot fix).
- “Is your insertion sort stable? Point at the character that makes it so.” — The `>` in `a[j] > current`: an equal key stops the walk.
- “Why is selection sort not stable, and what is it good for?” — The swap sends `a[i]` far right, past equal keys; at most $n - 1$ swaps, for when writes are expensive.
- “How many shifts does your insertion sort make on `[4, 1, 3, 9, 7, 2]`, and why that number?” — 7, the number of inversions: each shift fixes one.
- “When would you not use counting sort?” — When the range `k` is much larger than `n`, or the keys are not small non-negative integers.
- “Counting sort is $O(n + k)$. Doesn’t that break the $n \log n$ lower bound?” — No: the bound is for sorts that only compare; counting sort indexes by value.

Lab 10 — know-why questions

- “Why `<=` and not `<` in your merge — no test checks it?” — On a tie the left element, which came first in the input, must go first; `<` still sorts numbers but breaks stability.
- “Why `yield from sort(lo, mid)` and not just `sort(lo, mid)`?” — `sort` is a generator; calling it creates a generator object and runs nothing. `yield from` runs it and passes its snapshots up.
- “Why is the pivot in its final place after a partition?” — Everything smaller is to its left, everything else to its right, and sorting either side cannot move anything across it.

- “Your quicksort recurses on one side and loops on the other. Why, and which side?” — The smaller side is the call, so each frame covers at most half its caller’s range: at most $\log_2 n$ frames. Two calls would go n deep on sorted input with “first”.
- “What does your quicksort cost on sorted input with `pivot=“first”`, and how do you know?” — $n(n-1)/2$ comparisons: each partition peels off one element. Counted in Part 6.
- “Why does your build-heap loop run from $n // 2 - 1$ down to 0?” — Leaves are already heaps; a sift-down needs both subtrees to be heaps, so parents are done after their children.
- “Why is merge sort’s extra space $O(n)$, and quicksort’s $O(\log n)$?” — One scratch Array of n slots for the merges; quicksort only needs its call stack, which the smaller-side rule keeps at $\log_2 n$.

Lab 11 — know-why questions

- “Why does your insert stop one step early?” — Only the parent can be changed; once you hold None there is no link left to set.
- “Why is the empty tree’s height -1 ?” — So that a leaf is $1 + \max(-1, -1) = 0$ with no special case, and height counts edges.
- “Why is the parent-only check not enough? Show me a tree it gets wrong.” — The BST property is about whole subtrees: 9 as the right child of 3 in 8’s left subtree passes every parent check.
- “Why is your in-order walk sorted?” — Everything in a node’s left subtree is smaller and written first; everything in its right subtree larger and written after.
- “Why does `level_order` need a queue and not recursion?” — It visits by distance from the root, first found first visited; the call stack is last in, first out.
- “Why does the successor never have a left child — and why do you care?” — It was reached by going left until there was none; so removing it is always a leaf or one-child case.
- “What does your tree cost on sorted input, and what else breaks?” — Height $n - 1$, $O(n)$ per operation, and the recursive methods hit `RecursionError` at about 1,000 nodes.

Lab 12 — know-why questions

- “Why does push append at the end, and not insert at the right place?” — The end is the only slot that keeps the tree complete; the order is then repaired along one path, $O(\log n)$, where inserting in order would shift $O(n)$ items.
- “Why must sift-down swap with the better child?” — The child that moves up becomes the parent of its sibling, so it must beat the sibling as well as the item; only the better child does.
- “Your pop has a special case. What is it for?” — After removing the last item from a one-item heap there is no index 0 to write into; that item was the root, so return it.
- “Why does your `heapify` loop run backwards, and why is it $O(n)$ and not $O(n \log n)$?” — A sift-down needs both subtrees to be heaps already; and each node moves at most

its height, which is 0 for half the nodes, 1 for a quarter, and so on — the sum is at most n .

- “Why is there a counter in the priority queue’s triple?” — On equal priorities Python would compare the items, which may raise `TypeError`; the unique counter settles every tie first, and makes ties first-come first-served.
- “Where in your code does `MaxHeap` get its behaviour?” — Only from `_beats`; every comparison in `MinHeap` goes through it.

Lab 13 — know-why questions

- “Why does `hash( ' abc ' )` change between runs, and why is that a feature?” — string hashes use a secret key chosen at start-up, so an attacker cannot pick keys that all collide; within one run the hash never changes.
- “Your `_resize` cannot copy the buckets across. Why?” — the index is `hash % capacity`; with a new capacity every key has a new index, so a copied key sits where `get` does not look.
- “Why a tombstone and not `None`?” — `None` ends a search; emptying a slot that other keys probed past makes them unreachable.
- “Why does your `put` keep probing after it meets a tombstone?” — the key may be further along; inserting at the tombstone at once would store it twice.
- “Why do you count tombstones in the load check?” — only `None` stops a probe; without the count, tombstones can fill every free slot, and misses scan the whole table or never stop.
- “What is the worst case of your `get`, and can a `resize` fix it?” — $O(n)$, when keys share a bucket; no, keys with equal (or equal-modulo-every-capacity) hashes collide at every size.

Lab 14 — know-why questions

- “Why does your `add_node` check first — what would `add_edge` do without it?” — `add_edge` calls `add_node` on both ends; without the check, every new edge wipes the old edges of its endpoints.
- “Your BFS marks a node when it is enqueued. What goes wrong if you mark it when it is dequeued?” — It can be enqueued again while it waits, and appear twice in the order (A B C C D on the sample graph).
- “Why does your `dfs_iterative` push the neighbours in reverse, and why mark on pop?” — The stack returns the last push first; marking on pop lets a deeper route push a vertex again and be taken first, exactly as recursion does.
- “Why is BFS on a `MatrixGraph` $O(V^2)$ even for a sparse graph?” — Each neighbour calls scans a whole row of V cells.
- “In your directed `has_cycle`, why is a black neighbour not a cycle?” — It is finished and off the current path; an edge to it is only a second route (the diamond). Only a grey vertex is an ancestor on the path.
- “How does your `topological_sort` know there is a cycle?” — Fewer than V vertices came out: the rest wait on each other and never reach in-degree 0.

Lab 15 — know-why questions

- “Where in your code is the precedence of `*` over `+` decided?” — Nowhere as a number: `expr` calls `term`, and `*` is handled in `term`, so it ends up deeper in the tree.
- “Why a `while` in `expr` and not a recursive call?” — The loop makes the old tree the **left** child, which is left associativity; right recursion gets `1 - 2 - 3` wrong, and left recursion never terminates.
- “Why does unary minus call `factor`, not `expr`?” — So it binds only the next factor: `-5 + 2` is `(0 - 5) + 2`, not `-(5 + 2)`.
- “What does `parse` check after `expr` returns, and why?” — That no token is left; otherwise `3 + 4 )` would be accepted as 7.
- “What order does `evaluate` visit the nodes in, and where have you seen that order before?” — Post-order; it is the postfix list of Lab 06.
- “What limits how deeply your calculator can nest parentheses?” — Python’s recursion limit: three frames per pair, about 330 pairs.

6. Environment problems you will see

Symptom	Cause	Fix
python opens the Microsoft Store	Windows App Execution Aliases	<i>Settings</i> → <i>Apps</i> → <i>Advanced app settings</i> → <i>App execution aliases</i> : switch off <code>python.exe</code> and <code>python3.exe</code>
pytest is not recognised	venv not activated	<code>.\.venv\Scripts\activate</code> ; the prompt must start with <code>(.venv)</code>
activate refused: “running scripts is disabled”	PowerShell execution policy	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> , then activate again
<code>ModuleNotFoundError: No module named 'labs'</code>	running from the wrong folder	<code>cd</code> to the DSA27 folder — the one with <code>pytest.ini</code> in it
VS Code runs a different Python	wrong interpreter selected	<code>Ctrl+Shift+P</code> → <i>Python: Select Interpreter</i> , then <code>.\.venv\Scripts\python.exe</code>
Tests pass in the terminal but not in VS Code (or the reverse)	two interpreters	as above; <code>python -c "import sys; print(sys.prefix)"</code> in both
Changes to <code>labs/lab01.py</code> have no effect in the REPL	the module was already imported	<code>exit()</code> and restart the REPL, or just use <code>pytest</code>

Symptom	Cause	Fix
A test “hangs”	an infinite while loop, or $O(n^2)$ in <code>unique_in_order</code>	Ctrl+C; the traceback shows the line it was on
graphviz tests fail in <code>pytest -m "not challenge"</code>	the dot binary is not installed or not on PATH	install Graphviz from https://graphviz.org/download/ and reopen the terminal. Not needed for the labs themselves
<code>git clone</code> asks for a password	HTTPS cloning of a public repo should not — they probably mistyped the URL	copy the URL from the README

7. Answers to the take-home practice

The take-home items are ungraded and discussed at the start of the following lab. The quick answers:

- **Lab 01, item 3:** `x` is `[1, [2, 3, 4]]` — `y[0] = 9` changed only the copy; the inner list is shared.
- **Lab 01, item 5:** the time roughly doubles when the input doubles — linear.
- **Lab 02, item 3:** 20 iterations for 1,000,000 — one per binary digit, $\text{floor}(\log_2 n) + 1$.
- **Lab 02, item 4:** 7 guesses — $\text{ceil}(\log_2 101)$ — the same halving as Lab 01’s Checkpoint 4 and, in Week 8, binary search.
- **Lab 02, item 5:** yes, a straight line for primes: $O(n)$.
- **Lab 03, item 1:** sorting is $O(n \log n)$; counting (dictionary or Bag) is $O(n)$.
- **Lab 03, item 3:** `zip` stops at the shorter iterable, and a generator only computes a value when asked, so only ten are ever produced.
- **Lab 03, item 5:** slicing copies the rest of the list at every call, so the sliced version is $O(n^2)$ in total and the index version $O(n)$.

Lab 09 — take-home practice answers

- **Item 1 (W9-C1 to C5):** the worked code is in `week09-answers.md`, Part G, and `solutions/practice/week09.py`. The tests students fail on cost: `dutch_flag` written as an insertion sort (the one-pass test allows $3n$ reads on 3,000 values), and `sort_k_sorted` written with selection sort (it allows $n(k + 1) = 8,000$ comparisons; selection sort makes about 2 million, the reference 3,456). `sort_by_key` that compares items fails with `TypeError` on the Record test; `counting_sort_by_key` placed back to front fails the stability test.
- **Item 2 (W9-S1):** 6 passes, 27 comparisons, 16 swaps (16 inversions); after pass 5 the array is sorted, and pass 6 is the pass with no swap.
- **Item 3 (W9-T4):** bubble 12, selection 14, insertion 12 on `[5, 4, 3, 2, 1]`.
- **Item 4 (W9-B2):** 15 swaps against 3 on `[6, 5, 4, 3, 2, 1]`.
- **Item 5 (W9-E5):** the decision tree needs $n!$ leaves, so height $\geq \log_2 n!$; counting sort

does not compare, so the bound does not apply.

Lab 10 — take-home practice answers

- **Item 1 (W10-C1 to C5):** see the week 10 answers file, Part G. The comparison-counting tests fail for a `merge_k_sorted` that merges the lists one after another (about 131,000 comparisons on 64 lists of 64, against a budget of 28,672), a `count_inversions` that checks every pair, a `kth_smallest` that sorts first (about 44,000 against $8n = 32,768$), and a `merge_sort_bottom_up` that calls itself. `sort_colours` is checked by reads and writes (at most $3n$ and $2n$).
- **Item 2 (W10-T2):** `_partition` on `[5, 8, 1, 9, 3, 7, 2, 6]` swaps at `j = 0, 2, 4, 6`, ends as `[5, 1, 3, 2, 6, 7, 9, 8]` and returns **4**; 7 comparisons.
- **Item 3 (W10-S1):** merges in the order `a[0:2]`, `a[2:4]`, `a[0:4]`, `a[4:6]`, `a[6:8]`, `a[4:8]`, `a[0:8]`; after the third, `[1, 3, 5, 6, 8, 7, 2, 4]`; 14 comparisons in all.
- **Item 4 (W10-B2):** `[1, 2, 3, 4, 5]` — the upward build leaves `[3, 5, 1, 4, 2]`, with 3 above 5, and the sort returns `[1, 2, 4, 5, 3]`.
- **Item 5 (W10-K2):** $T(n) = T(n - 1) + (n - 1)$, so $T(n) = n(n - 1)/2$ exactly; the stack stays $O(1)$ with the smaller-side rule, $n - 1$ frames without it.

Lab 11 — take-home practice answers

- **Item 1 (W11-C1 to C5):** the worked code is in `week11-answers.md`, Part G, and `solutions/practice/week11.py`. The tests that count reads are the ones students fail: `is_balanced` with a separate height at every node (the naive version reads 212,992 times on 8,191 nodes against a budget of 32,768), and `range_values` as a full in-order walk (49,149 reads against 400).
- **Item 2 (W11-S1):** the final tree after the four deletes has pre-order 60, 25, 10, 30, 35, 65 and height 3. `delete(45)` is the successor-is-the-right-child case (successor 60, whose child 65 moves up).
- **Item 3 (W11-T3):** (a) post-order 5, 12, 15, 10, 25, 40, 30, 20;
(b) post-order D, E, B, F, C, A.
- **Item 4 (W11-B3):** `delete(20)` on Q, or `delete(8)` on the sample tree: the successor is the node's own right child. On Q, 17 is lost and 25 appears twice.
- **Item 5 (W11-K2):** 3,145,726 calls on a right chain of 20 nodes ($3 \cdot 2^n - 2$), against 41 for a correct height.

Lab 12 — take-home practice answers

- **Item 1 (W12-C1 to C5):** see the week 12 answers file, Part G. The two comparison-counting tests fail for a `top_k` that sorts everything (about $14n$ comparisons against a budget of $3n$) and for a `merge_sorted` that puts all values in the heap at once.
- **Item 2 (W12-T1):** after the pushes `[1, 2, 3, 6, 5, 8]`; the pops return 1 and 2, leaving `[2, 5, 3, 6, 8]` and then `[3, 5, 8, 6]`.
- **Item 3 (W12-T2):** `[0, 1, 2, 3, 4, 5, 8, 7, 6, 9]`, **5 swaps**, 14 comparisons; the first sift-down is at index 4.

- **Item 4 (W12-B2):** popping [2, 4, 3, 9, 7, 8, 5, 12, 10] leaves [4, 9, 3, 10, 7, 8, 5, 12] — 4 above 3 — and the next pop returns 4.
- **Item 5 (W12-K3):** [5, 4, 3, 2, 1] becomes [3, 1, 5, 2, 4], with 1 below 3. Any input whose minimum starts deeper than level 1 fails the same way.

Lab 13 — take-home practice answers

- **Item 1 (W13-C1 to C5):** each is one pass with a ChainingHashMap: value → first index; item → seen; sorted letters → group position; character → last index with a window start that only moves forward; prefix sum → count, seeded with 0 → 1. The large tests (20,000 or more elements) run in well under a second on the reference; an $O(n^2)$ answer takes tens of seconds or more.
- **Item 2 (W13-S1):** resize to 8 on put 7; final buckets 2: 18 · 3: 3 · 6: 14 · 7: 7, size 4.
- **Item 3 (W13-S2):** final slots 6 14 _ _ _ † 29 21, size 4, one tombstone; put 29 reuses the tombstone left by 13, put 14 cannot reach the one left by 5.
- **Item 4 (W13-B2):** entry = entry.next after entry.next was overwritten; save next first. With 1, 5, 9, 2 into capacity 4, keys 5 and 1 are lost.
- **Item 5 (W13-E2):** only slow, never wrong — == still tells keys apart; every operation $O(n)$, building $O(n^2)$.

Lab 14 — take-home practice answers

- **Item 1 (W14-C1 to C5):** the code is in week14-answers.md, Part G, and in solutions/practice/week14.py; pytest --solutions tests/test_practice_week14.py gives 30 passed. The usual failure is count_islands with a recursive flood fill: RecursionError on the 2,000-cell island, and nothing else fails.
- **Item 2 (W14-T1, T3):** BFS on H from A is A B D C E F G, queue B D, D C E, C E, E F, F, G, empty. The iterative DFS is A B C F E D G; D and E are each pushed twice.
- **Item 3 (W14-S2):** the edge F → D proves the cycle (D is grey); D → C reaches a black vertex and is harmless. topological_sort gives None, and A B D C E F without F → D.
- **Item 4 (W14-B1):** A B D C E E F F F G G G on H.
- **Item 5 (W14-K2):** one list per vertex instead of per component, and $\Theta(V(V + E))$ on a connected graph instead of $\Theta(V + E)$.

Lab 15 — take-home practice answers

- **Item 1 (W15-C1 to C5):** the worked code is in week15-answers.md, Part G, and solutions/practice/week15.py (52 tests). Where students fail: to_infix_minimal that parenthesises only lower precedence — it prints the tree of 1 - (2 - 3) as 1 - 2 - 3 and fails both the string test and the round-trip test; postfix_to_tree that pops left before right (the trees come out mirrored, assert BinOp('+', BinOp('*', ... against the expected tree); calculate_with_power with a loop for ^ (gives 64 for 2 ^ 3 ^ 2) or with unary below power (gives 4 for -2 ^ 2). The C3 round-trip test uses the student's own Parser, so it can only pass once the lab is finished.

- **Item 2 (W15-S1):** 10, 18, 4, 6, 8; the right-recursive tree for $9 - 3 - 2$ is $9 - (3 - 2) = 8$.
- **Item 3 (W15-T2):** 11 calls, deepest point 6 frames of the three methods; tree $(2 * (3 - 1)) + 4$, value 8.0.
- **Item 4 (W15-B1):** any chain of $-$ (or mixed $-$ and $+$) at one level: $10 - 4 - 3$ gives 9, $5 - 1 + 2$ gives 2. A chain of $+$ alone gives the right value.
- **Item 5 (W15-K2):** nested parentheses — parser about 3k, evaluate 1; flat chain — parser constant, evaluate about k; balanced — both logarithmic. Measured limits from a script: 329 pairs, 997 numbers.

8. Worked solution notes, labs 09–15

What the key lines of each reference solution do, and why they are written that way. The code itself is in `solutions/`, and in the appendix of the TAs' edition of the lab manual.

Lab 09 — worked solution notes

The reference is `solutions/dsa/sorting.py`. The lines worth explaining:

- **`_copy(values)`** is `Array.from_values(values)`: the caller's list is never touched, and the working storage obeys the storage rule. **`_finish(steps)`** runs a generator to the end with `for state, _ in steps: pass` (starting from `state = []`) and returns `list(state)` — one copy, at the end.
- **The snapshot parameter.** Every basic `_steps` function is `def bubble_sort_steps(values, snapshot=list)` and yields `snapshot(a)`. The default copies — right for an animation. The plain form is `_finish(bubble_sort_steps(values, snapshot=_live))`, where `_live(a)` returns a itself. Without it, the first version of the reference took over a minute to sort 800 values: a list copy before each of $\sim n^2/2$ comparisons is $O(n^3)$. Students' generators without the parameter still pass every test; Part 7 of the lab is where they add it.
- **Bubble.** `for end in range(n - 1, 0, -1)` — `a[end+1:]` is in place; `swapped = False` at the top of each pass; `yield ... (j, j + 1)` **before** the comparison; `break` when a pass swapped nothing.
- **Selection.** `smallest = i`, then `if a[j] < a[smallest]: smallest = j` with a `yield` before each comparison; after the scan, `if smallest != i: swap and yield (i, smallest)`. Strict `<` picks the first of equal minima — which does not make it stable; the swap is what breaks stability.
- **Insertion.** `while j >= 0 and a[j] > current: shift a[j + 1] = a[j], j -= 1, and a[j + 1] = current` inside the loop, then `yield`. The inner write of `current` is only for the animation — every frame a permutation — and doubles the writes (the counts figure shows 498,241 writes for 248,621 inversions). The textbook version writes `current` once, after the loop; both are accepted.
- **Counting.** Empty list first; a `ValueError` for anything that is not a non-negative int; `k = (max(values) if max_value is None else max_value) + 1`; `counts = Array(k, fill=0)`; then the write-out loop over `range(k)` with an output index

- i. It is not stable in any visible sense: it rebuilds integers from counts. The stable, record-moving version is W9-C5.

Lab 10 — worked solution notes

The reference is the second half of `solutions/dsa/sorting.py`. The lines that carry the week:

- **_merge** — `i, j, k = lo, mid, lo`: `k` starts at `lo`, not 0, so `scratch[k]` lines up with `a[k]` and one shared scratch Array serves every merge. `if a[i] <= a[j]`: is the stability. Two leftover loops, of which only one ever runs, then `for k in range(lo, hi): a[k] = scratch[k]` — the copy back that the next merge up the tree reads.
- **merge_sort_steps** — `scratch = Array(len(a))` once, outside sort. `if hi - lo <= 1`: return is the base case for both 0 and 1 elements. `yield` from `sort(lo, mid)` and `yield` from `sort(mid, hi)` before `_merge(a, lo, mid, hi, scratch)`, then one frame highlighting `tuple(range(lo, hi))`. The half-open `[lo, hi)` makes `mid` belong to the right half with `no ± 1` anywhere.
- **_choose_pivot** — returns an **index**. "random" uses the `rng` made once in `quick_sort_steps` as `random.Random(27)`, so the frames repeat. The median of three is two chained comparisons per candidate, `first <= middle <= last` or `last <= middle <= first`, then the same for `first`, else `hi` — no `sorted()`.
- **_partition** — `pivot = a[hi]`, `boundary = lo`, the loop over `range(lo, hi)` (the pivot is excluded), the strict `a[j] < pivot`, and the final `a[boundary], a[hi] = a[hi], a[boundary]`; it returns `boundary`.
- **quick_sort_steps** — `a[p], a[hi] = a[hi], a[p]` before `_partition` is the line that makes the strategy matter. `while lo < hi`: with `if q - lo < hi - q`: recursing left and setting `lo = q + 1`, else recursing right and setting `hi = q - 1`: the smaller side is the call, the larger is the loop. On sorted input with "first" the call is always on an empty range.
- **_sift_down** — `largest = index`, then left and right each compared with `a[largest]` — which picks the larger child and checks it against the parent in one step — and `if largest == index`: return.
- **heap_sort_steps** — `for index in range(n // 2 - 1, -1, -1)` builds the heap; `for end in range(n - 1, 0, -1)` swaps `a[0]` with `a[end]` and sifts with `size = end`. The same backwards loop opens `heapify` in Week 12.

The snapshot cost (Part 8). As written, the plain forms run the `_steps` generators to the end, and every `yield list(a)` copies `n` values: merge sort measured 0.36, 1.29 and 4.7 seconds for 1,000, 2,000 and 4,000 values — quadratic. The basic sorts in the reference already take a `snapshot=list` parameter, with the plain form passing a no-copy function; the same change to the three advanced sorts brings those times to about 0.07, 0.13 and 0.27 seconds. Students who finish early: have them make that change, then time it.

Lab 11 — worked solution notes

The reference is `solutions/dsa/tree.py`. The lines worth explaining:

- **insert looks ahead.** Inside the loop, if `node.left` is `None`: `node.left = new`; return, and the mirror on the right. The loop only ever steps onto a child that exists, so the parent is always in hand when the empty spot is found. Duplicates return at `if value == node.value`. The empty tree is handled before the loop.
- **contains, min, max are loops**, not recursions: `node = node.left` if `value < node.value` else `node.right`. No stack, so they work on a chain of any length.
- **height and size are two-line post-order recursions** through private helpers `_height(node)` and `_size(node)`; `_height(None)` is `-1`.
- **_is_valid(node, low, high)** uses `None` for “no bound” and tests `low is not None` and `node.value <= low` (and the mirror): strict, so a duplicate of an ancestor is rejected. The two recursive calls are joined by `and`, so it stops at the first failure.
- **The traversals** create `out = []` in the public method, call `self._in_order(self.root, out)`, and return `out`. The three helpers differ only in where `out.append(node.value)` sits.
- **level_order** uses `CircularQueue()` with the default capacity and relies on its growth; it holds nodes, not values, and returns `[]` early for an empty tree.
- **delete in one pass, with no recursion.** It walks down keeping `parent, node`. If the node has two children it walks to the successor with `parent, successor = node, node.right` and then `successor.left` until `None` — note that `parent` starts at **the node itself**, which is exactly what makes the successor-is-the-right-child case work — copies `node.value = successor.value`, and sets `node = successor`. From there one block handles every case: `child = node.left` if `node.left` is not `None` else `node.right`, then `self.root = child` if `parent` is `None`, else `parent.left = child` if `parent.left` is `node`, else `parent.right = child`. The identity test `is`, not a value comparison, decides which link to change.

Lab 12 — worked solution notes

The reference is `solutions/dsa/heap.py`. The lines that carry the week:

- **_sift_up** — while `index > 0`: then `parent = (index - 1) // 2`. The condition keeps `index 0` from computing the `parent -1`, which the `DynamicArray` would silently read as the last item. The early return when the item does not beat its parent is what makes an average push $O(1)$: most new values stop at once.
- **_sift_down** — `best = index`, then if `left < size` and `self._beats(items[left], items[best])` and the same for `right`, **against `items[best]`**. That second comparison chooses the better child and checks it against the item in one step. if `best == index`: return ends the loop; otherwise swap and `index = best`. `size` is read once, before the loop: the array does not change length during a sift.
- **pop** — `last = self._items.pop()` first, **then** if `len(self._items) == 0`: return `last`. Removing the last item before looking at the root handles the one-item heap without a special index case; `best = self._items[0]` is saved before `self._items[0] = last` overwrites it.
- **heapify** — for `index` in `range(len(self._items) // 2 - 1, -1, -1)`. The start is the last parent, the stop `-1` includes the root, and the step `-1` is the whole correctness argument. The same line opens `heap_sort_steps` in solu-

tions/dsa/sorting.py.

- **is_valid** — loops over **children** from 1, comparing each with items[(child - 1) // 2] through **_beats**: n - 1 comparisons, one per edge.
- **PriorityQueue.enqueue** — self._heap.push((priority, self._counter, item)) then self._counter += 1. dequeue and peek check is_empty() and return [2] of the triple. Nothing in the class sifts; the heap does all the work.

Students who finish early: ask them to add a replace(value) to MinHeap — pop the root and push value in one sift-down (overwrite index 0, then sift down), and say why it saves a sift-up. heapq.heapreplace is that method.

Lab 13 — worked solution notes

solutions/dsa/hashmap.py, the lines that carry the week:

- **ChainingHashMap.put**: self._buckets[i] = Entry(key, value, self._buckets[i]) is push_front in one line — the new entry's next is the old head. It comes **after** the walk that returns early on an existing key, so a key is never stored twice. The resize test self._size / len(self._buckets) > self.max_load runs after the insert, with >, so 6 keys in 8 buckets (exactly 0.75) do not resize and the 7th does.
- **ChainingHashMap.delete**: prev, entry = None, self._buckets[i] and prev, entry = entry, entry.next keep the two references in step; the if prev is None branch is the head case, where the Array slot itself is rewritten.
- **ChainingHashMap._resize**: nxt = entry.next is saved before entry.next = self._buckets[i] overwrites it. The existing nodes are relinked — no new Entry objects — and _size is untouched. self._index uses the new capacity because self._buckets was replaced first.
- **OpenAddressingHashMap.put**: the load check counts self._size + self._tombstones + 1; if that is over the limit, it doubles only when (self._size + 1) / capacity is too, and otherwise calls self._resize(capacity) at the same size to sweep tombstones. The probe loop records first_free at the first tombstone **or** at the terminating None, overwrites and returns if it meets the key, and breaks at None. After the loop, if self._keys[first_free] is TOMBSTONE: self._tombstones -= 1 keeps the count right when a tombstone is reused. Because the load check keeps at least one None in the table, the loop always sets first_free.
- **_find** (a helper, not in the skeleton): shared by get and delete; returns -1 at the first None, skips TOMBSTONE with slot is not TOMBSTONE and slot == key, and returns -1 if every slot was probed.
- **OpenAddressingHashMap.delete**: self._keys[i] = TOMBSTONE, self._values[i] = None (so the value can be freed), _size -= 1, _tombstones += 1.
- **OpenAddressingHashMap._resize**: resets both counters, then re-inserts each live key at the **first None** of its probe sequence in the new table — safe because the new table has no tombstones and no duplicates — and counts _size back up.
- **The skeleton change** this week: the given ChainingHashMap.__contains__ was self.get(key, None) is not None, which reports a key stored with the value None as absent (W13-B1). It now calls get(key) and catches KeyError. OpenAd-

dressingsHashMap gained the same `__contains__`, an `__iter__`, a `_tombstones` counter in `__init__`, and docstrings for `put` and `get`. Week 14's Graph relies on `in` and iteration over a `ChainingHashMap`.

Lab 14 — worked solution notes

`solutions/dsa/graph.py`, the lines that carry the week:

- **Graph.__init__ keeps _order** beside `_adjacent`. A hash map returns its keys in bucket order, randomised for strings between runs, so `nodes()` returns `list(self._order)`; that is what makes every trace in the lecture, the lab and the question bank reproducible. This is a skeleton change made this week (the attribute is given to the students in `__init__`).
- **add_node**: `if node not in self._adjacent`: guards both the `put` of an empty `DynamicArray` and the `_order.append`. **add_edge** calls it on both ends, appends `target` to the source's array, and appends `source` to the target's only if `not self.directed and source != target` — a loop is stored once.
- **neighbours** is `list(self._adjacent.get(node))`: the hash map's `get` raises `KeyError` for a missing node, and the list is a copy, so callers cannot change the adjacency by mutating the result.
- **Graph.edges**: walks `_order`; a `ChainingHashMap` `done` holds the nodes already walked; `(node, neighbour)` is reported when the graph is directed or the neighbour is not yet done; `done.put(node, True)` comes **after** the inner loop, so a loop edge `(A, A)` is still reported once.
- **MatrixGraph.add_node**: `self._index.put(node, len(self._order))` before `_order.append`, then `row.append(False)` for every existing row, then a new row of `len(self._order)` `Falses`. **edges** uses `first = 0` if `self.directed` else `i`, the upper triangle.
- **bfs**: `visited.put(start, True)` before the loop and `visited.put(neighbour, True)` immediately before `queue.enqueue(neighbour)`. `order` is the only Python list, and it is the result.
- **_dfs_visit** marks, appends, and recurses on unvisited neighbours — four lines. **dfs_iterative**: `if node in visited`: `continue` after the `pop`, then mark and append, then for `i in range(len(neighbours) - 1, -1, -1)` pushing unvisited neighbours.
- **shortest_path_unweighted**: `parent.put(start, None)` makes the parent map the visited set; the goal test is on **dequeue**, so `start == goal` returns `[start]` at once; the walk back is `while node is not None`, then `path.reverse()`. It relies on `ChainingHashMap.__contains__` catching `KeyError` rather than comparing the value with `None` (Week 13's skeleton fix).
- **connected_components** reuses `bfs` for each unseen node and marks every member in seen.
- **has_cycle**: `_undirected_cycle_from(graph, node, parent, visited)` with `elif neighbour != parent`: `return True`; `_directed_cycle_from` with the module constants `WHITE, GREY, BLACK = 0, 1, 2`, `colour.get(neighbour, WHITE)` (absent means white), `return True` on grey, and `colour.put(node,`

BLACK) before return False. Both helpers are recursive, so both inherit Python's 1,000-frame limit; the tests' graphs are small.

- **topological_sort**: raises ValueError for an undirected graph; in-degree map initialised to 0 for every node, then +1 per edge target; ready nodes in a CircularQueue in nodes() order; if len(order) < len(graph): return None. On the lecture's prerequisites graph it returns the twelve courses in the numbered order of the figure.

Lab 15 — worked solution notes

The reference is solutions/dsa/translation.py. The lines worth explaining:

- **tokenize** is one while i < len(text) loop with four branches: ch.isspace() skips; ch in OPERATORS or ch in "(" appends one character; ch in DIGITS or ch == "." starts a number, and the inner loop is while i < len(text) and (text[i] in DIGITS or text[i] == "."), counting the points in points; then number = text[start:i], rejected if points > 1 or number == ".". The else raises ValueError(f"unexpected character {ch!r} at position {i}"). DIGITS is the string "0123456789", deliberately not str.isdigit, which accepts Arabic-Indic and other Unicode digits.
- **parse** is three steps: if self.peek() is None: raise ValueError("empty expression"); tree = self.expr(); if self.peek() is not None: raise. The first check is redundant — factor would raise on empty input anyway — but it gives the better message.
- **expr / term** are the lecture's code: node = self.term(), then while self.peek() in ("+", "-"): op = self.advance(); node = BinOp(op, node, self.term()). peek() returning None at the end needs no special case: None in ("+", "-") is False.
- **factor** tests token is None first, then "(" (advance, self.expr(), require ")", advance), then "-" (advance, BinOp("-", Num(0.0), self.factor())), then a number, recognised by token[0] in DIGITS or token[0] == "." — the lexer guarantees that anything starting that way is a valid number — and finally raises on anything else.
- **evaluate** returns node.value for a Num; otherwise computes left = evaluate(node.left) and right = evaluate(node.right) **before** looking at node.op — post-order — then four ifs and a final raise ValueError. / raises ZeroDivisionError by itself.
- **calculate** is tokens = tokenize(text), tree = Parser(tokens).parse(), return evaluate(tree).
- **Storage rule**: the parser's tokens are in the course Array (given in __init__), the tree is node objects, and tokenize's result list is the interface. evaluate_postfix (Week 6) is unchanged and uses the student's Stack.
- **Depth**: measured from a script, 329 nested pairs parse and 330 raise RecursionError (in the parser); a flat chain of 997 numbers evaluates and 998 raise (in evaluate, whose recursion follows the left-leaning tree). A student who asks how to remove the limit: evaluate with an explicit stack in post-order, or raise sys.setrecursionlimit — both beyond the tests.